



Sailboat
This is

UML y Patrones

2ª Edición

Larman

Este manual presenta una introducción práctica al análisis y diseño orientado a objetos (A/D OO) y a aspectos relacionados de desarrollo iterativo.

El objetivo principal del libro es ayudar a los estudiantes y a los desarrolladores a crear diseños orientados a objetos mediante la aplicación de un conjunto de principios y heurísticas explicables.

El libro va dirigido a desarrolladores y estudiantes con experiencia en un lenguaje de programación orientado a objetos, pero con poca experiencia en el análisis y diseño orientados a objetos, así como a estudiantes de informática o cursos de ingeniería del software que estudien la tecnología de objetos y a aquellas personas que, familiarizadas con el A/D OO, quieran aprender la notación UML, aplicar patrones, o mejorar y perfeccionar sus habilidades de análisis y diseño.

Esta nueva edición presenta mejoras importantes respecto a la anterior, tales como:

- Se actualizan los casos de uso para seguir el enfoque de [Cockburn01] que ha tenido bastante aceptación.
- Se utiliza el conocido Proceso Unificado (*Unified Process, UP*) como ejemplo de proceso iterativo. Por tanto, todos los artefactos se nombran de acuerdo con la terminología del UP, como Modelo de Dominio.
- Tratamiento actualizado de los patrones de diseño.
- Introducción al análisis arquitectural.
- Introducción de Variaciones Protegidas como una extensión GRASP.
- Actualización de la notación UML más reciente.
- Discusión de algunos aspectos prácticos de dibujo utilizando pizarras o herramientas CASE para UML.



LibroSite es una página web asociada al libro, con una gran variedad de recursos y material adicional tanto para los profesores como para estudiantes. Apoyos, docencia, ejercicios de autocontrol, enlaces relacionados, material de investigación, etc., hacen de LibroSite el complemento académico perfecto para este libro.

www.librosite.net/larman

PEARSON
Educación

www.pearsoneducacion.com

ISBN 84-205-3438-2

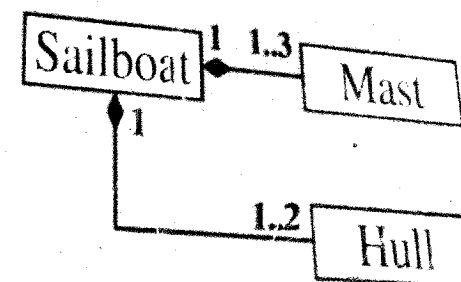
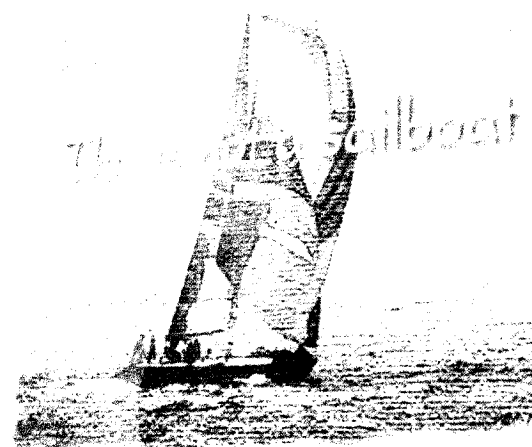


9 788420 534381

2ª Edición

UML y Patrones

Una introducción



This is not a sailboat

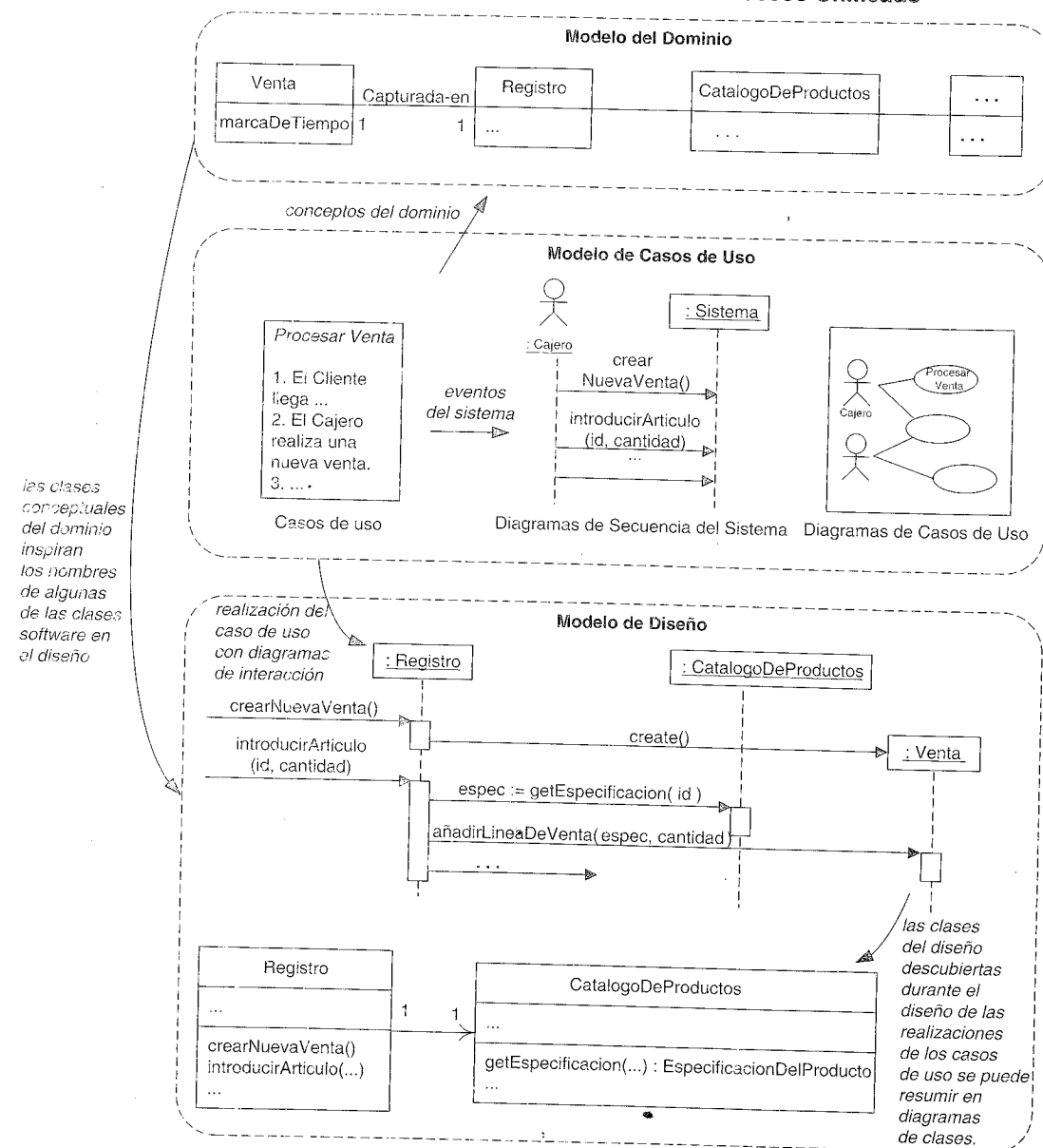


Craig Larman

Muestra de los artefactos del Proceso Unificado y evolución temporal (c-comenzar; r-refinar)

Disciplina	Artefacto Iteración →	Inicio I1	Elab. E1...En	Const. C1...Cn	Trans. T1...T2
Modelado del Negocio	Modelo del Dominio		c		
Requisitos	Modelo de Casos de Uso	c	r		
	Visión	c	r		
	Especificación Complementaria	c	r		
	Glosario	c	r		
Diseño	Modelo de Diseño		c	r	
	Documento de Arquitectura SW		c		
	Modelo de Datos		c	r	
Implementación	Modelo de Implementación		c	r	r
Gestión del Proyecto	Plan de Desarrollo SW	c	r	r	r
Pruebas	Modelo de Pruebas		c	r	
Entorno	Marco de Desarrollo	c	r		

Muestra de las relaciones entre los artefactos del Proceso Unificado



Patrones de Software Generales para la Asignación de Responsabilidades (GRASP)

Patrón	Descripción
Experto en Información	¿Un principio general del diseño de objetos y la asignación de responsabilidades? Asigne una responsabilidad al experto en información, —la clase que tiene la información necesaria para llevar a cabo la responsabilidad.
Creador	¿Quién crea? (Nótese que la Factoría es una solución alternativa frecuente.) Asigne a la clase B la responsabilidad de crear una instancia de la clase A si se cumple alguno de los puntos siguientes: 1. B contiene a A 2. B agrega a A 3. B tiene los datos de inicialización de A 4. B registra a A 5. B utiliza estrechamente a A
Controlador	¿Quién gestiona un evento del sistema? Asigne la responsabilidad de gestionar un mensaje de un evento del sistema a una clase que represente una de estas opciones: 1. Representa el sistema global, dispositivo o un subsistema (controlador de fachada). 2. Representa un escenario de caso de uso en el que tiene lugar el evento del sistema (controlador de caso de uso o sesión).
Bajo Acoplamiento (evaluativo)	¿Cómo dar soporte a las bajas dependencias y al incremento de la reutilización? Asigne responsabilidades de manera que el acoplamiento (innecesario) se mantenga bajo.
Alta Cohesión (evaluativo)	¿Cómo mantener manejable la complejidad? Asigne responsabilidades de manera que la cohesión permanezca alta.
Polimorfismo	¿Quién es el responsable cuando el comportamiento varía en función del tipo? Cuando las alternativas o comportamientos relacionados varían según el tipo (clase), asigne la responsabilidad del comportamiento —utilizando operaciones polimórficas— a los tipos para los que varía el comportamiento.
Fabricación Pura	¿Quién es el responsable cuando está desesperado, y no quiere violar los principios de alta cohesión y bajo acoplamiento? Asigne un conjunto altamente cohesivo de responsabilidades a una clase de “comportamiento” artificial o de conveniencia que no representa un concepto del dominio del problema —algo inventado—, para dar soporte a la alta cohesión, bajo acoplamiento y la reutilización.
Indirección	¿Cómo asignar responsabilidades para evitar el acoplamiento directo? Asigne la responsabilidad a un objeto intermedio para mediar entre otros componentes o servicios, de manera que no se acoplan directamente.
Variaciones Protegidas	¿Cómo asignar responsabilidades a los objetos, subsistemas, y sistemas de manera que las variaciones o inestabilidad en estos elementos no influya de manera no deseable en otros elementos? Identifique los puntos de variaciones predecibles o inestabilidad; asigne las responsabilidades para crear una “interfaz” estable alrededor de ellos.

24045

PRÉSTAMO 48 HORAS

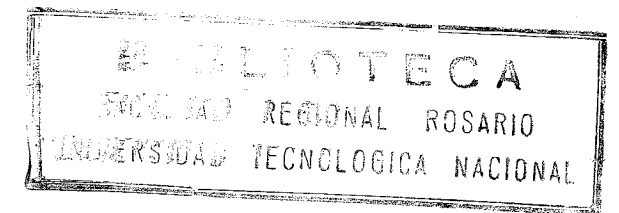
Fecha de devolución	
07/04/08.	21 ABR 2009
14/04/08	28/05/09
28/05/08	19/06/09
05/05/08	10/08/09
13/05/08	13/08/09
01 SEP 2008	18/08/09
01 DIC 2008	7 DEC 2008
15/02/08	7/9/09
07 MAR 2009	25/09/09
27/3/09	29/09/09
30/3/09	14 NOV 2009
06/04/09	19/11/09
14/04/09	17 MAR 2010
17 ABR 2009	
Evite suspensiones : devuelva en término	

UML Y PATRONES

UNA INTRODUCCIÓN AL ANÁLISIS
Y DISEÑO ORIENTADO A OBJETOS
Y AL PROCESO UNIFICADO

Segunda edición

24045



UML Y PATRONES

UNA INTRODUCCIÓN AL ANÁLISIS
Y DISEÑO ORIENTADO A OBJETOS
Y AL PROCESO UNIFICADO

Segunda edición

UTN Facultad Regional Rosario Biblioteca

Autor: Larman, C

Uml Y Patrones

Email: biblioteca@frro.utn.edu.ar

Tel.: 4481871 Int. 111



24045

CRAIG LARMAN

Traducción:

Begoña Moros Valle

Universidad de Murcia

Supervisión de la traducción y revisión técnica:

Jesús García Molina

Universidad de Murcia

BIBLIOTECA	
Fecha:	08-03-05
Registro:	7167
Inventario:	24045
Sig. Top:	681.3 6294
Ejemplar:	1 (uso) 2a. ed.
Proc.:	compuer

PEARSON
Prentice
Hall

Madrid • México • Santafé de Bogotá • Buenos Aires • Caracas • Lima
Montevideo • San Juan • San José • Santiago • São Paulo • White Plains

LARMAN, C.

UML Y PATRONES. Una introducción al análisis y diseño orientado a objetos y al proceso unificado. Segunda edición

PEARSON EDUCACIÓN, S.A., Madrid, 2003

ISBN: 84-205-3438-2

Materia: Informática 681.3

Formato 195 × 250

Páginas: 624

Todos los derechos reservados.

Queda prohibida, salvo excepción prevista en la ley, cualquier forma de reproducción, distribución, comunicación pública y transformación de esta obra sin contar con autorización de los titulares de propiedad intelectual. La infracción de los derechos mencionados puede ser constitutiva de delito contra la propiedad intelectual (*arts. 270 y sgts. Código Penal*).

DERECHOS RESERVADOS

© 2003 respecto a la segunda edición en español por:

PEARSON EDUCACIÓN, S.A.

Ribera del Loira, 28

28042 Madrid (España)

LARMAN, C.

UML Y PATRONES. Una introducción al análisis y diseño orientado a objetos y al proceso unificado. Segunda edición

ISBN: 84-205-3438-2

Depósito Legal: M.23.390-2004

Última reimpresión, 2004

PRENTICE HALL es un sello editorial autorizado de PEARSON EDUCACIÓN, S.A.

Traducido de:

APPLYING UML AND PATTERNS: An Introduction to Object-Oriented Analysis and Design and the Unified Process, Second edition, by Craig Larman.

Published by Pearson Education, Inc. Publishing as Prentice Hall PTR.

Prentice Hall, Inc.

© 2002 All rights reserved.

ISBN: 0-13-092569-1

Edición en español:

Equipo editorial:

Editor: David Fayerman Aragón

Técnico editorial: Ana Isabel García

Equipo de producción:

Director: José Antonio Clares

Técnico: José Antonio Hernán

Diseño de cubierta: equipo de diseño de PEARSON EDUCACIÓN, S.A.

Composición: COPIBOOK, S.L.

Impreso por: LVEL, S. A.

IMPRESO EN ESPAÑA - PRINTED IN SPAIN

Para Julie

Sin su apoyo, esto no habría sido posible.

Para Haley y Hannah

Gracias por aguantar a un papá distraído, ¡otra vez!

RESUMEN DE CONTENIDO

Parte 1: Introducción

1. Análisis y diseño orientado a objetos	3
2. Desarrollo iterativo y el Proceso Unificado	13
3. Caso de estudio: el sistema de punto de venta NuevaEra	27

Parte 2: Inicio

4. Inicio	33
5. Comprensión de los requisitos	39
6. Modelo de Casos de Uso: escritura de requisitos en contexto	43
7. Identificación de otros requisitos	79
8. Del inicio a la elaboración	103

Parte 3: Elaboración en la Iteración 1

9. Modelo de Casos de Uso: representación de los diagramas de secuencia del sistema	113
10. Modelo del Dominio: visualización de conceptos	121
11. Modelo del Dominio: añadir asociaciones	145
12. Modelo del Dominio: añadir atributos	157
13. Modelo de Casos de Uso: añadir detalles con los contratos de las operaciones	167
14. De los requisitos al diseño en esta iteración	181
15. Notación de los diagramas de interacción	185
16. GRASP: diseño de objetos con responsabilidades	201
17. Modelo de Diseño: realización de los casos de uso con los patrones GRASP	231
18. Modelo de Diseño: determinación de la visibilidad	261

19.	Modelo de Diseño: creación de los diagramas de clases de diseño	267
20.	Modelo de Implementación: transformación de los diseños en código ..	281

Parte 4: Elaboración en la Iteración 2

21.	La Iteración 2 y sus requisitos	291
22.	GRASP: más patrones para asignar responsabilidades	305
23.	Diseño de las realizaciones de casos de uso con los patrones de diseño GoF ..	321

Parte 5: Elaboración en la Iteración 3

24.	La Iteración 3 y sus requisitos	359
25.	Relaciones entre casos de uso	361
26.	Modelado de la generalización	369
27.	Refinamiento del modelo del dominio	383
28.	Añadir nuevos DSSs y contratos	401
29.	Modelado del comportamiento con diagramas de estado	407
30.	Diseño de la arquitectura lógica con patrones	417
31.	Organización de los paquetes de los modelos de diseño e implementación ..	443
32.	Introducción al análisis arquitectural y el SAD	451
33.	Diseño de más realizaciones de casos de uso con objetos y patrones	473
34.	Diseño de un framework de persistencia con patrones	501

Parte 6: Temas especiales

35.	Sobre el dibujo de diagramas y las herramientas	531
36.	Introducción a cuestiones relacionadas con la planificación iterativa y el proyecto	539
37.	Comentarios acerca del desarrollo iterativo y el UP	553
38.	Más notación UML	567

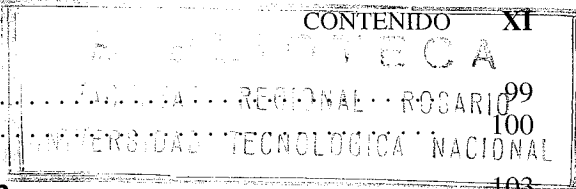
Presentación	XIX
Prólogo	XXI

Parte 1: Introducción

Capítulo 1: Análisis y diseño orientado a objetos	3
Aplicación de UML y patrones en el A/DOO	3
Asignación de responsabilidades	5
¿Qué es análisis y diseño?	6
¿Qué son el análisis y diseño orientados a objetos?	6
Un ejemplo	7
UML	10
Lecturas adicionales	10
Capítulo 2: Desarrollo iterativo y el Proceso Unificado	13
La idea más importante del UP: desarrollo iterativo	14
Conceptos y buenas prácticas del UP adicionales	18
Las fases del UP y términos orientados a la planificación	19
Las disciplinas del UP (eran flujos de trabajo)	20
Adaptación del proceso y el Marco de Desarrollo	22
El UP ágil	23
El ciclo de vida “en cascada” secuencial	24
No se entendió el UP cuando... ..	25
Lecturas adicionales	25
Capítulo 3: Caso de estudio: el sistema de punto de venta NuevaEra ..	27
El sistema de punto de venta NuevaEra	27
Capas arquitectónicas y el énfasis del caso de estudio	28
Estrategia del libro: aprendizaje y desarrollo iterativo	29

Parte 2: Inicio

Capítulo 4: Inicio	33
Inicio: una analogía	34
La fase de inicio podría ser muy breve	35
¿Qué artefactos podrían crearse en la fase de inicio?	35
No se entendió la fase de inicio cuando...	36
Capítulo 5: Comprensión de los requisitos	39
Tipos de requisitos	40
Lecturas adicionales	41
Capítulo 6: Modelo de Casos de Uso: escritura de requisitos en contexto	43
Objetivos e historias	44
Antecedentes	44
Casos de uso y valor añadido	45
Casos de uso y requisitos funcionales	46
Tipos de casos de uso y formatos	46
Ejemplo completo: Procesar Venta	47
Explicación de las secciones	52
Objetivos y alcance de un caso de uso	56
Descubrimiento de actores principales, objetivos y casos de uso	60
Enhorabuena: se han escrito los casos de uso y no son perfectos	64
Escritura de casos de uso en un estilo esencial independiente de la interfaz de usuario	65
Actores	67
Diagramas de casos de uso	68
Requisitos en contexto y lista de características de bajo nivel	70
Los casos de uso no son orientados a objetos	71
Casos de uso en el UP	72
Caso de estudio: casos de uso en la fase de inicio de NuevaEra	76
Lecturas adicionales	76
Artefactos UP y contexto del proceso	77
Capítulo 7: Identificación de otros requisitos	79
Ejemplos del PDV NuevaEra	80
Ejemplo NuevaEra: Especificación Complementaria (Parcial)	80
Comentario: Especificación Complementaria	84
Ejemplo NuevaEra: Visión (Parcial)	87
Comentario: Visión	90
Ejemplo NuevaEra: un Glosario (Parcial)	94
Comentario: Glosario (Diccionario de Datos)	95
Especificaciones fiables: ¿un Oxímoron?	96
Artefactos disponibles en el sitio web del proyecto	97
¿Poco UML durante la fase de inicio?	97
Otros artefactos de requisitos en el UP	97



Lecturas adicionales	99
Artefactos UP y contexto del proceso	100
Capítulo 8: Del inicio a la elaboración	103
Punto de control: ¿qué sucedió en el inicio?	104
En la elaboración	105
Planificación de la siguiente iteración	106
Requisitos y énfasis de la iteración 1: habilidades de A/DOO fundamentales	108
¿Qué artefactos podrían crearse en la elaboración?	108
No se entendió la elaboración cuando...	110

Parte 3: Elaboración en la Iteración 1

Capítulo 9: Modelo de Casos de Uso: representación de los diagramas de secuencia del sistema	113
Comportamiento del sistema	114
Diagramas de secuencia del sistema	114
Ejemplo de un DSS	115
DSS entre sistemas	115
DSS y los casos de uso	116
Eventos del sistema y los límites del sistema	116
Asignación de nombres a los eventos y operaciones	117
Mostrar el texto del caso de uso	118
Los DSS y el Glosario	118
DSS en el UP	118
Lecturas adicionales	119
Artefactos del UP	120
Capítulo 10: Modelo del Dominio: visualización de conceptos	121
Modelos del Dominio	122
Identificación de las clases conceptuales	126
Clases conceptuales candidatas para el dominio de ventas	129
Guías para el modelado del negocio	130
Resolución de clases conceptuales similares: Registro vs. "TPDV"	132
Modelado del mundo <i>irreal</i>	133
Clases conceptuales de especificación o descripción	133
Notación UML, modelos y métodos: perspectivas múltiples	136
Reducción del salto en la representación	138
Ejemplo: el Modelo del Dominio del PDV NuevaEra	140
Modelos del Dominio en el UP	140
Lecturas adicionales	142
Artefactos del UP	142
Capítulo 11: Modelo del Dominio: añadir asociaciones	145
Asociaciones	145
Notación de las asociaciones en UML	146

Localización de las asociaciones—lista de asociaciones comunes	147
Guías para las asociaciones	148
Roles	149
¿Cómo de detalladas deben ser las asociaciones?	150
Asignación de nombres a las asociaciones	151
Múltiples asociaciones entre dos tipos	152
Asociaciones e implementación	152
Asociaciones del Modelo del Dominio del PDV NuevaEra	153
Modelo del Dominio del PDV NuevaEra	154
Capítulo 12: Modelo del Dominio: añadir atributos	157
Atributos	157
Notación de los atributos en UML	158
Tipos de atributos válidos	158
Clases de tipos de datos no primitivos	160
Deslizarse al diseño: ningún atributo como clave ajena	162
Modelado de cantidades y unidades de los atributos	162
Atributos en el Modelo del Dominio de NuevaEra	163
Multiplicidad de la LineaDeVenta al Artículo	163
Conclusión del Modelo del Dominio	164
Capítulo 13: Modelo de Casos de Uso: añadir detalles con los contra- tos de las operaciones	167
Contratos	167
Ejemplo de contrato: introducirArticulo	168
Secciones del contrato	169
Postcondiciones	169
Discusión: postcondiciones de introducirArticulo	171
La escritura de los contratos da lugar a actualizaciones en el Modelo del Dominio ..	172
¿Cuándo son útiles los contratos? ¿Contratos vs. casos de uso?	173
Guías: contratos	173
Ejemplo del PDV NuevaEra: contratos	174
Cambios en el Modelo del Dominio	175
Contratos, operaciones y UML	176
Contratos de las operaciones en el UP	177
Lecturas adicionales	178
Capítulo 14: De los requisitos al diseño en esta iteración	181
Iterativamente hacer lo correcto, y hacerlo correcto	181
¿No lleva eso semanas en hacerse? No, no exactamente	182
Pasar al diseño de objetos	182
Capítulo 15: Notación de los diagramas de interacción	185
Diagramas de secuencia y colaboración	186
Ejemplo de diagrama de colaboración: realizarPago	187
Ejemplo de diagrama de secuencia: realizarPago	187
Los diagramas de interacción son importantes	188

Notación general de los diagramas de interacción	189
Notación básica de los diagramas de colaboración	190
Notación básica de los diagramas de secuencia	195
Capítulo 16: GRASP: diseño de objetos con responsabilidades	201
Responsabilidades y métodos	202
Responsabilidades y los diagramas de interacción	203
Patrones	204
GRASP: Patrones de Principios Generales para Asignar Responsabilidades ..	205
Notación del diagrama de clases UML	206
Experto en Información (o Experto)	207
Creador	211
Bajo Acoplamiento	214
Alta Cohesión	217
Controlador	221
Diseño de objetos y tarjetas CRC	229
Lecturas adicionales	230
Capítulo 17: Modelo de Diseño: realización de los casos de uso con los patrones GRASP	231
Realizaciones de casos de uso	232
Comentarios sobre los artefactos	232
Realizaciones de los casos de uso para la iteración de NuevaEra	236
Diseño de objetos: crearNuevaVenta	236
Diseño de objetos: introducirArticulo	239
Diseño de objetos: finalizarVenta	243
Diseño de objetos: realizarPago	247
Diseño de objetos: ponerEnMarcha	252
Conexión de la capa de UI con la capa del dominio	255
Realizaciones de casos de uso en el UP	258
Resumen	258
Capítulo 18: Modelo de Diseño: determinación de la visibilidad	261
Visibilidad entre objetos	261
Visibilidad	262
Representación de la visibilidad en UML	266
Capítulo 19: Modelo de Diseño: creación de los diagramas de clases de diseño	267
Cuándo crear los DCD	267
Ejemplo de DCD	268
Terminología del DCD y el UP	268
Clases del Modelo de Dominio vs. clases del Modelo de Diseño	269
Creación de un DCD del PDV NuevaEra	269
Notación para los detalles de los miembros	277
DCD, dibujo y herramientas CASE	278
DCD en el UP	278
Artefactos del UP	280

Capítulo 20: Modelo de Implementación: transformación de los diseños en código	281
Programación y el proceso de desarrollo	282
Transformación de los diseños en código	284
Creación de las definiciones de las clases a partir de los DCDs	284
Creación de métodos a partir de los diagramas de interacción	287
Clases contenedoras/colecciones en el código	288
Manejo de excepciones y de errores	289
Definición del método Venta--crearLineaDeVenta	289
Orden de implementación	290
Programar probando primero	291
Resumen de la transformación de los diseños en código	292
Introducción a la solución del programa	292

Parte 4: Elaboración en la Iteración 2

Capítulo 21: La Iteración 2 y sus requisitos	299
Énfasis de la Iteración 2: diseño de objetos y patrones	299
De la Iteración 1 a la 2	300
Requisitos de la Iteración 2	301
Refinamiento de los artefactos orientados al análisis en esta iteración	303
Capítulo 22: GRASP: más patrones para asignar responsabilidades	305
Polimorfismo	306
Fabricación Pura	308
Indirección	312
Variaciones Protegidas	313
Capítulo 23: Diseño de las realizaciones de casos de uso con los patrones de diseño GoF	321
Adaptador (GoF)	322
Descubrimientos del "análisis" durante el diseño: Modelo del Dominio	324
Factoría (GoF)	326
Singleton (GoF)	328
Conclusiones del problema de los servicios externos con diversas interfaces	331
Estrategia (GoF)	332
Composite (GoF) y otros principios de diseño	337
Fachada (GoF)	346
Observador/Publicar-Suscribir/Modelo de Delegación de Eventos (GoF)	348
Conclusión	356
Lecturas adicionales	356

Parte 5: Elaboración en la Iteración 3

Capítulo 24: La Iteración 3 y sus requisitos	359
Requisitos de la Iteración 3	359
Énfasis de la Iteración 3	359

Capítulo 25: Relaciones entre casos de uso	361
La relación de inclusión (<i>include</i>)	362
Terminología: casos de uso concretos, abstractos, base y adicional	364
La relación de extensión (<i>extend</i>)	365
La relación de generalización (<i>generalize</i>)	366
Diagramas de casos de uso	367
Capítulo 26: Modelado de la generalización	369
Nuevos conceptos para el Modelo del Dominio	369
Generalización	371
Definición de superclases y subclases conceptuales	373
Cuándo definir una clase conceptual	375
Cuándo definir una superclase conceptual	377
Jerarquías de clases conceptuales del PDV NuevaEra	377
Clases conceptuales abstractas	380
Modelado de los cambios de estado	381
Jerarquías de clases y herencia en el software	382
Capítulo 27: Refinamiento del modelo del dominio	383
Clases asociación	383
Agregación y composición	385
Intervalos de tiempo y precios de los productos: arreglar un "error" de la Iteración 1	390
Nombres de los roles de asociación	390
Roles como conceptos vs. roles en asociaciones	391
Elementos derivados	392
Asociaciones calificadas	393
Asociaciones reflexivas	394
Elementos ordenados	394
Utilización de paquetes para ordenar el Modelo del Dominio	394
Capítulo 28: Añadir nuevos DSSs y contratos	401
Nuevos diagramas de secuencia del sistema	401
Nuevas operaciones del sistema	403
Nuevos contratos de operaciones del sistema	404
Capítulo 29: Modelado del comportamiento con diagramas de estados	407
Eventos, estados y transiciones	407
Diagramas de estados	408
¿Diagramas de estados en el UP?	409
Diagramas de estados de casos de uso	409
Diagramas de estados de casos de uso para la aplicación del PDV	410
Clases que se benefician de los diagramas de estados	411
Representación de eventos externos e internos	412
Notación adicional de los diagramas de estados	413
Lecturas adicionales	415

Capítulo 30: Diseño de la arquitectura lógica con patrones	417
Arquitectura del software	418
Patrón de arquitectura: Capas (<i>Layers</i>)	420
Principio de Separación Modelo-Vista	440
Lecturas adicionales	442
Capítulo 31: Organización de los paquetes de los modelos de diseño e implementación	443
Guías para la organización de paquetes	444
Notación adicional de los paquetes en UML	450
Lecturas adicionales	450
Capítulo 32: Introducción al análisis arquitectural y el SAD	451
Análisis arquitectural	452
Tipos y vistas de la arquitectura	454
La ciencia: identificación y análisis de los factores de la arquitectura	454
Ejemplo: tabla de factores parcial de la arquitectura del PDV NuevaEra	457
El arte: resolución de los factores de la arquitectura	459
Resumen de los temas del análisis arquitectural	466
Análisis arquitectural en el UP	467
Lecturas adicionales	471
Capítulo 33: Diseño de más realizaciones de casos de uso con objetos y patrones	473
Mantenimiento de los servicios ante los fallos mediante servicios locales; rendimiento con el almacenamiento local	473
Manejo de fallos	479
Mantenimiento de los servicios ante los fallos mediante un Proxy (GoF)	484
Diseño para los requisitos no funcionales o de calidad	488
Acceso a los dispositivos físicos externos con adaptadores; comprar vs. construir ..	488
Factoría Abstracta (GoF) para familias de objetos relacionados	490
Gestión de pagos con Polimorfismo y Hacerlo Yo Mismo	493
Conclusión	498
Capítulo 34: Diseño de un framework de persistencia con patrones ..	501
El problema: objetos persistentes	502
La solución: un servicio de persistencia a partir de un framework de persistencia ..	502
Frameworks	503
Requisitos para el servicio y framework de persistencia	503
Ideas claves	504
Patrón: Representación de Objetos como Tablas	504
Perfil (<i>Profile</i>) de modelado de datos en UML	505
Patrón: Identificador de Objeto	505
Acceso al servicio de persistencia con una Fachada	506
Correspondencia de los objetos: patrón Conversor (<i>Mapper</i>) de Base de Datos o Intermediario (<i>Broker</i>) de Base de Datos	507

Diseño del framework con el patrón Método Plantilla	509
Materialización con el patrón Método Plantilla	510
Configuración de conversores con una FactoríaDeConversores	515
Patrón: Gestión de Caché	515
Reunir y ocultar sentencias SQL en una clase	516
Estados transaccionales y el patrón Estado	517
Diseño de una transacción con el Patrón Command	520
Materialización perezosa con un Proxy Virtual	522
Cómo representar las relaciones en tablas	524
Superclase ObjetoPersistente y separación de intereses	525
Cuestiones sin resolver	526

Parte 6: Temas especiales

Capítulo 35: Sobre el dibujo de diagramas y las herramientas	529
Diseño especulativo y razonamiento visual	529
Sugerencias para dibujar los diagramas de UML en el proceso de desarrollo	530
Herramientas y características de ejemplo	533
Ejemplo dos	534
Capítulo 36: Introducción a cuestiones relacionadas con la planificación iterativa y el proyecto	537
Priorización de los requisitos	538
Priorización de los riesgos del proyecto	541
Planificación adaptable vs. predictiva	541
Planes de Fase y de Iteración	543
Plan de Iteración: ¿qué hacemos en la siguiente iteración?	543
Traza de los requisitos a través de las iteraciones	544
La (in)validez de las primeras estimaciones	546
Organización de los artefactos del proyecto	547
Algunas cuestiones de la planificación de la iteración del equipo	548
No se entendió la planificación en el UP cuando... ..	549
Lecturas adicionales	550
Capítulo 37: Comentarios acerca del desarrollo iterativo y el UP ...	551
Buenas prácticas y conceptos del UP adicionales	551
Las fases de construcción y transición	553
Otras prácticas interesantes	554
Motivos para fijar la duración de una iteración	555
El ciclo de vida secuencial en "cascada"	555
Ingeniería de usabilidad y diseño de interfaces de usuario	561
El Modelo de Análisis del UP	561
El producto del RUP	562
Los desafíos y mitos de la reutilización	563

Capítulo 38: Más notación UML	565
Notación general	565
Diagramas de implementación	566
Clase plantilla (parametrizada, genérica)	567
Diagramas de actividades	567
Bibliografía	571
Glosario	577
Índice alfabético	583

PRESENTACIÓN

Programar es divertido, pero desarrollar software de calidad es difícil. Entre las ideas espléndidas, los requisitos o la “visión”, y un producto software funcionando, hay mucho más que programar. El análisis y el diseño que definen cómo solucionar el problema, qué programar, y la expresión de este diseño de forma que sea fácil de comunicar, revisar, implementar y evolucionar constituyen la parte central de este libro. Esto es lo que aprenderás.

El Lenguaje Unificado de Modelado (UML) se ha convertido en el lenguaje aceptado universalmente para los planos del diseño software. UML es el lenguaje visual utilizado a lo largo de este libro para ilustrar las ideas de diseño, poniendo énfasis en cómo aplican los desarrolladores realmente los elementos UML utilizados con más frecuencia, más que en características oscuras del lenguaje.

La importancia de los patrones en la creación de sistemas complejos es reconocida desde hace tiempo en otras disciplinas. Los patrones de diseño software son los que nos permiten describir fragmentos de diseño y reutilizar ideas de diseño, ayudando a beneficiarse de la experiencia de otros. Los patrones dan nombre y forma a heurísticas abstractas, reglas y buenas prácticas de técnicas orientadas a objetos. Ningún ingeniero razonable quiere partir de una pizarra en blanco, y este libro ofrece una paleta de patrones de diseño que pueden utilizarse fácilmente.

Pero el diseño de software parece un poco árido y misterioso cuando no se presenta en el contexto de un proceso de ingeniería del software. Y sobre este tema, estoy encantado de que para su segunda edición, Craig Larman haya elegido adoptar e introducir el Proceso Unificado, mostrando cómo puede aplicarse de un modo relativamente simple y poco ceremonioso. Al presentar el caso de estudio a través de un proceso centrado en la arquitectura, dirigido por el riesgo e iterativo, los consejos de Craig tienen un contexto realista; expone la dinámica de lo que ocurre en realidad en el desarrollo de software, y muestra las fuerzas externas que entran en juego. Las actividades del diseño están conectadas a otras tareas, y ya no aparecen como actividades puramente cerebrales de transformaciones sistemáticas o intuición creativa. Y Craig y yo estamos convencidos de los beneficios de un desarrollo iterativo, que verás ilustrado con todo detalle a lo largo del libro.

Así es que para mí, este libro tiene la mezcla correcta de ingredientes. Aprenderás un método sistemático para abordar el Análisis y Diseño Orientado a Objetos (A/DOO) de un gran profesor, un metodologista brillante, y un “gurú de la OO” que lo ha enseñado a miles de personas alrededor del mundo. Craig describe el método en el contexto del Pro-

ceso Unificado. Presenta gradualmente patrones de diseño más sofisticados —esto hará el libro muy útil y fácil de usar cuando te enfrentes con retos de diseño del mundo real. Y utiliza la notación más ampliamente aceptada.

Me siento honrado de haber tenido la oportunidad de trabajar directamente con el autor de este importante libro. Disfruté leyendo la primera edición, y me encantó que me pidiera que revisara el borrador de su segunda edición. Nos encontramos varias veces e intercambiamos muchos correos electrónicos. He aprendido mucho de Craig, incluso sobre nuestro propio proceso de trabajo en el Proceso Unificado y cómo mejorarlo y aplicarlo en varios contextos organizacionales. Estoy seguro de que aprenderás mucho, bastante, al leer este libro, incluso si ya estás familiarizado con el A/DOO. Y, como yo, a menudo volverás a él, para refrescar tu memoria o para comprender mejor las explicaciones y experiencias de Craig.

En un proceso iterativo, el resultado de la segunda iteración mejora la primera. Similarmemente, la escritura madura, supongo; incluso si tienes la primera edición, disfrutarás y te beneficiarás de la segunda.

¡Feliz lectura!

*Philippe Kruchten
Rational Fellow
Rational Software Canada
Vancouver, BC*

*Diseñar sistemas
de objetos robustos
y de fácil
mantenimiento*

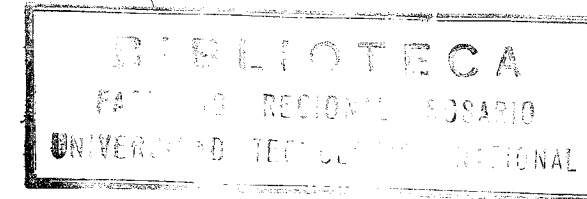
*Seguir un mapa
a través de los
requisitos, análisis,
diseño
y codificación*

*Usar UML para
ilustrar los modelos
de análisis y diseño*

*Mejorar los diseños
aplicando los
patrones de diseño
GRASP y
los de la "pandilla
de los cuatro"*

*Aprendizaje
eficiente
siguiendo una
presentación
refinada*

*Aprendizaje
mediante
un ejercicio
realista*



PRÓLOGO

¡Gracias por leer este libro! Ésta es una introducción práctica al análisis y diseño orientado a objetos (A/DOO), y a aspectos relacionados de desarrollo iterativo. Estoy agradecido de que la primera edición fuese recibida por todo el mundo como una introducción sencilla al A/DOO, traducida a muchos idiomas. Por tanto, esta segunda edición refina y se construye sobre el contenido de la primera, más que reemplazarla. Quiero dar las gracias sinceramente a todos los lectores de la primera edición.

Aquí tienes los beneficios que te proporcionará el libro.

Primero, el uso de la tecnología de objetos ha proliferado en el desarrollo de software, y el dominio del A/DOO es crítico para crear sistemas de objetos robustos y de fácil mantenimiento.

Segundo, si eres nuevo en el A/DOO, te preguntarás comprensiblemente cómo avanzar por este tema tan complejo; este libro presenta un mapa bien definido —el Proceso Unificado— de manera que te puedas mover en un proceso paso a paso, desde los requisitos al código.

Tercero, el Lenguaje Unificado de Modelado (UML) ha emergido como la notación estándar para el modelado; por tanto, te resultará útil familiarizarte con él. Este libro enseña las técnicas del A/DOO utilizando la notación UML.

Cuarto, los patrones de diseño comunican los estilos y soluciones consideradas como “buenas prácticas”, que los expertos en el diseño orientado a objetos utilizan para la creación de sistemas. En este libro aprenderás a aplicar patrones de diseño, incluyendo los populares patrones de la “pandilla de los cuatro” (*gang-of-four*) y los patrones GRASP que comunican los principios fundamentales de asignación de responsabilidades en el diseño orientado a objetos. Aprender y aplicar patrones acelerará el dominio del análisis y el diseño.

Quinto, la estructura y el enfoque del libro se basa en años de experiencia en la enseñanza y asesoramiento a miles de personas en el arte del A/DOO. Refleja esa experiencia proporcionando un enfoque eficiente, probado y refinado para aprender la materia de manera que se optimiza tu inversión en leer y aprender.

Sexto, examina de manera exhaustiva un único caso de estudio —para ilustrar de manera realista el proceso de A/DOO completo—, y examina en profundidad detalles espinosos del problema; es un ejercicio realista.

Traducción a código

Séptimo, muestra cómo obtener código Java a partir de los artefactos del diseño de objetos.

Diseño de una arquitectura de capas

Octavo, explica cómo diseñar una arquitectura de capas y relaciona la capa de interfaz gráfica de usuario con las capas del dominio y servicios técnicos.

Diseño de un framework

Por último, muestra cómo diseñar un framework orientado a objetos aplicándolo a la creación de uno para el almacenamiento persistente en una base de datos.

Objetivos

El objetivo global es:

Ayudar a los estudiantes y a los desarrolladores a crear diseños orientados a objetos mediante la aplicación de un conjunto de principios y heurísticas explicables.

Estudiando y aplicando la información y las técnicas que se presentan aquí, adquirirás experiencia en la comprensión de un problema en términos de sus procesos y conceptos, y en el diseño de una solución utilizando objetos.

A quiénes está dirigido este libro

Este libro es una *introducción* al A/DOO, al análisis de requisitos relacionado, y al desarrollo iterativo, con el Proceso Unificado como ejemplo de un proceso; la intención no es ser un texto avanzado. Va destinado a la siguiente audiencia:

- Desarrolladores y estudiantes con experiencia en un lenguaje de programación orientado a objetos, pero que son nuevos —o relativamente nuevos— en el análisis y diseño orientado a objetos.
- Estudiantes de informática o cursos de ingeniería del software que estudien la tecnología de objetos.
- Aquellos familiarizados con el A/DOO que quieran aprender la notación UML, aplicar patrones, o que quieran mejorar y perfeccionar sus habilidades de análisis y diseño.

Requisitos

Se asumen —y se necesitan— algunos conocimientos previos para aprovechar este libro:

- Conocimiento y experiencia en un lenguaje de programación orientado a objetos como Java, C#, C++ o Smalltalk.
- Conocimiento de los conceptos generales de la tecnología de objetos, como clase, instancia, interfaz, polimorfismo, encapsulación y herencia.

No se definen los conceptos fundamentales de la tecnología de objetos.

Ejemplos Java

En general, el libro presenta ejemplos de código en Java o plantea implementaciones Java, debido a que su uso está muy extendido. Sin embargo, las ideas presentadas son aplicables a la mayoría —si no a todos— los lenguajes orientados a objetos.

Organización del libro

La estrategia global en la organización de este libro es introducir las cuestiones de análisis y diseño en un orden similar al de un proyecto de desarrollo de software a través de una fase de “inicio” (término del Proceso Unificado) seguido por tres iteraciones (ver Figura P.1).

1. Los capítulos de la fase de inicio introducen los fundamentos del análisis de requisitos.
2. La iteración 1 introduce el A/DOO básico y cómo asignar responsabilidades a los objetos.
3. La iteración 2 se centra en el diseño de objetos, especialmente en introducir algunos “patrones de diseño” muy utilizados.
4. La iteración 3 introduce una variedad de temas, como el análisis de la arquitectura y el diseño de frameworks.

El Libro

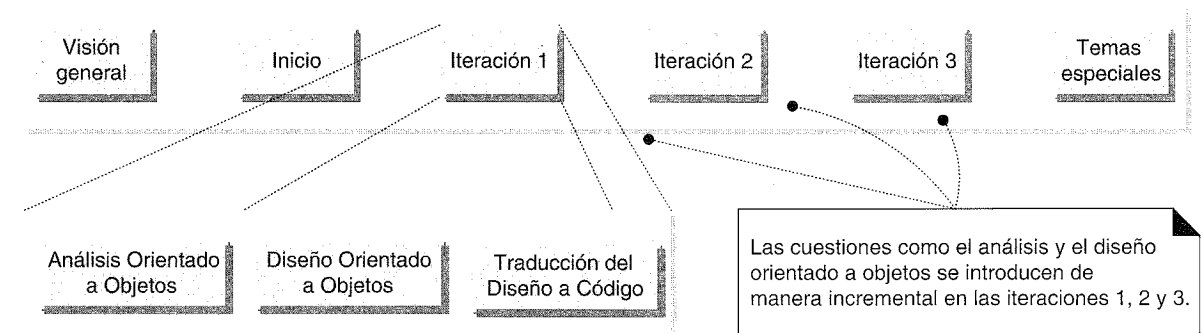


Figura P.1. El libro está organizado como un proyecto de desarrollo.

Recursos web relacionados

- Por favor, consúltese www.craiglarman.com para artículos relacionados con la tecnología de objetos, patrones y procesos.
- Se pueden encontrar recursos para los profesores en www.phptr.com/larman.

Mejoras a la primera edición

Aunque mantiene el mismo núcleo central que la primera edición, la segunda ha refinado muchos aspectos como:

- Se actualizan los casos de uso para seguir el enfoque de [Cockburn01] que ha tenido bastante aceptación.
- Se utiliza el bien conocido Proceso Unificado (UP) como ejemplo de proceso iterativo dentro del cual introducir el A/DOO. Por tanto, todos los artefactos se nombran de acuerdo con la terminología del UP, como Modelo de Dominio.
- Nuevos requisitos en el caso de estudio, que nos encaminan a la tercera iteración.
- Tratamiento actualizado de los patrones de diseño.
- Introducción al análisis arquitectural.
- Introducción de Variaciones Protegidas como un patrón GRASP.
- Balance al 50% entre los diagramas de secuencia y colaboración.
- Actualizada a la notación UML más reciente.
- Discusión de algunos aspectos prácticos de dibujo utilizando pizarras o herramientas CASE para UML.

Reconocimientos

En primer lugar, un agradecimiento muy especial a mis amigos y colegas de Valtech, desarrolladores de objetos a nivel mundial y expertos en el desarrollo iterativo, quienes de alguna manera contribuyeron a apoyar, o revisar el libro, incluyendo a Chris Tarr, Michel Ezran, Tim Snyder, Curtis Hite, Celso González, Pascal Roques, Ken DeLong, Brett Schuchert, Ashley Johnson, Chris Jones, Thomas Liou, Darryl Gebert, Frank Rodorigo, Jean-Yves Hardy, y muchos más que no puedo nombrar.

A Philippe Kruchten por escribir la presentación del libro, revisar y ayudarme de muchas formas.

A Martin Fowler y Alistair Cockburn por muchas discusiones esclarecedoras sobre procesos y diseños, comentarios y revisiones.

A John Vlissides y Cris Kobryn por sus amables comentarios.

A Chelsea Systems y John Gray por ayudarme con algunos requisitos inspirados en su sistema PDV con tecnología Java ChelseaStore.

A Pete Coad y Dave Astels de TogetherSoft por su apoyo.

Muchas gracias a los otros revisores, incluyendo Steve Adolph, Bruce Anderson, Len Bass, Gary K. Evans, Al Goerner, Luke Hohmann, Eric Lefebvre, David Nunn, y Robert J. White.

Gracias a Paul Becker de Prentice-Hall por creer que la primera edición sería un proyecto que merecía la pena, y a Paul Petralia y Patti Guerrieri por dirigir la segunda.

Por último, agradezco especialmente a Graham Glass por abrir una puerta.

Semblanza del autor

Craig Larman trabaja como Director de Procesos para Valtech, una compañía consultora internacional con sucursales en Europa, Asia y América del Norte, especializada en

desarrollo de sistemas de negocio electrónico, tecnologías de objetos y desarrollo iterativo con el Proceso Unificado.

Desde mediados de los ochenta, Craig ha ayudado a miles de desarrolladores a aplicar el análisis, diseño y programación orientada a objetos, y a organizaciones a adoptar las prácticas del desarrollo iterativo.

Después de una carrera fracasada como músico callejero, construyó sistemas en APL, PL/I, y CICS en los setenta. Desde comienzos de los ochenta —después de una completa recuperación— comenzó a interesarse por la inteligencia artificial (teniendo poca propia), procesamiento del lenguaje natural y representación del conocimiento, y construyó sistemas de conocimiento con máquinas Lisp, Prolog y Smalltalk. Toca mal la guitarra eléctrica en su banda *Los Requisitos Cambiantes* (antes se llamaba *Los Requisitos*, pero algunos miembros de la banda cambiaron...) a la que dedica su tiempo libre.

Es licenciado en informática por la Universidad de Simon Fraser en Vancouver, Canadá.

Contacto

Se puede contactar con Craig en clarman@ieee.org y www.craiglarman.com. Serán bienvenidas las preguntas de los lectores y profesores, y peticiones de conferencias, asesoramiento y consultoría.

Convenciones tipográficas

Esto es un **término nuevo** en una frase. Esto es un nombre de *Clase* o un *método* en una frase. Esto es una referencia a un autor [Bob67]. El operador de resolución de alcance independiente del lenguaje “--” se utiliza para indicar una clase y su método asociado como sigue: *NombreClase--NombreMetodo*.

Notas de producción

El manuscrito de este libro se creó con Adobe FrameMaker. Todos los dibujos se hicieron con Microsoft Visio. La fuente del texto es New Century Schoolbook. Las imágenes impresas finales se generaron con ficheros PDF utilizando Adobe Acrobat Distilled, a partir del PostScript generado por un controlador AGFA.

Parte 1

INTRODUCCIÓN

ANÁLISIS Y DISEÑO ORIENTADO A OBJETOS

El cambio de enfoque (a patrones) tendrá un efecto profundo y duradero en el modo en el que escribimos los programas.

Ward Cunningham y Ralph Johnson

Objetivos

- Comparar y contrastar análisis y diseño.
 - Definir análisis y diseño orientado a objetos (A/DOO).
 - Ilustrar con un pequeño ejemplo.
-

1.1. Aplicación de UML y patrones en el A/DOO

¿Qué significa tener un buen diseño de objetos? Este libro es una herramienta para ayudar a los desarrolladores y a los estudiantes a aprender las habilidades fundamentales en el análisis y diseño orientado a objetos (A/DOO). Estas técnicas son esenciales para la creación de software de fácil mantenimiento, robusto y bien definido, utilizando las tecnologías y lenguajes de objetos como Java, C++ , Smalltalk y C#.

El proverbio “tener un martillo no te hace un arquitecto” es especialmente cierto con respecto a la tecnología de objetos. Conocer un lenguaje orientado a objetos (como Java) es necesario pero un primer paso insuficiente para crear sistemas de objetos. También es preciso conocer cómo “pensar en objetos”.

Esto es una introducción al A/DOO mientras aplicamos el Lenguaje Unificado de Modelado (UML), patrones y el Proceso Unificado. No se debe entender como un texto avanzado; se centra en el dominio de los fundamentos, como el modo de asignar responsabilidades a los objetos, la notación UML utilizada con más frecuencia, y patrones de

*Esto es una
introducción*

diseño comunes. Al mismo tiempo, principalmente en los capítulos posteriores, el material progresa a unos pocos temas de nivel intermedio, tal como el diseño de frameworks.

Utilización
de UML

El libro no trata únicamente de UML. UML es una notación visual estándar. Tan útil como aprender una notación, hay más cosas orientadas a objetos importantes que aprender; concretamente, cómo pensar en objetos —cómo diseñar sistemas orientados a objetos—. UML no es A/DOO o un método, es simplemente una notación. No es tan útil aprender a hacer diagramas UML sintácticamente correctos y quizás una herramienta CASE para UML, si no se es capaz de crear un diseño excelente o evaluar y mejorar uno existente. Ésta es la habilidad más difícil y valiosa. En consecuencia, este libro es una introducción al diseño de objetos.

Con todo necesitamos un lenguaje para el A/DOO y los “planos de software”, tanto como una herramienta para pensar, como una forma de comunicación con otros. Por tanto, explora cómo *aplicar* UML al servicio del A/DOO, y cubre la notación UML utilizada con más frecuencia. Se presta especial atención a ayudar a que la gente aprenda el arte y la ciencia de construir sistemas de objetos, más que en la notación.

Aplicación
de patrones
y asignación de
responsabilidades

¿Cómo deberíamos asignar las **responsabilidades** a las clases de objetos? ¿Cómo deberían interaccionar los objetos? ¿Qué clases deberían hacer qué? Éstas son preguntas claves en el diseño de un sistema. Ciertas soluciones contrastadas a problemas de diseño se pueden expresar (y se han expresado) como principios, heurísticas o **patrones** de buenas prácticas —llamados fórmulas de solución de problemas que codifican principios de diseño ejemplares—. Este libro, al enseñar cómo aplicar patrones, favorece un aprendizaje más rápido y un uso propio de experto de estos estilos de diseño de objetos fundamentales.

Un caso
de estudio

Esta introducción al A/DOO se ilustra con un **único caso de estudio** que se sigue a lo largo del libro, profundizando en el análisis y diseño lo suficiente para tener en cuenta, y solucionar, algunos de los horribles detalles que se deben considerar y solucionar en un problema real.

Casos de uso
y análisis
de requisitos

El A/DOO (y todo el diseño software) está fuertemente relacionado con la actividad que es un requisito previo del **análisis de requisitos**, que incluye escribir **casos de uso**. Por tanto, el caso de estudio comienza con una introducción a este tema, a pesar de que realmente no es orientado a objetos.

Un ejemplo
de proceso
iterativo,
el Proceso
Unificado

Dadas muchas posibles actividades desde los requisitos hasta la implementación, ¿cómo debería proceder un desarrollador o un equipo? El análisis de requisitos y el A/DOO requieren que se presenten en el contexto de algún proceso de desarrollo. En este caso, se utiliza como *ejemplo* de **proceso de desarrollo iterativo** el bien conocido **Proceso Unificado**, en cuyo marco se presentan estos temas. Sin embargo, los temas de análisis y diseño que se cubren son comunes a muchos enfoques, y aprenderlos en el contexto del Proceso Unificado no invalida su aplicabilidad a otros métodos.

En conclusión, este libro ayuda a los estudiantes y desarrolladores:

- A aplicar principios y patrones para crear mejores diseños de objetos.
- A seguir un conjunto de actividades comunes en el análisis y diseño, basado en el Proceso Unificado como un ejemplo.
- A crear los diagramas utilizados con más frecuencia en la notación UML.

Esto se ilustra en el contexto de un único caso de estudio.

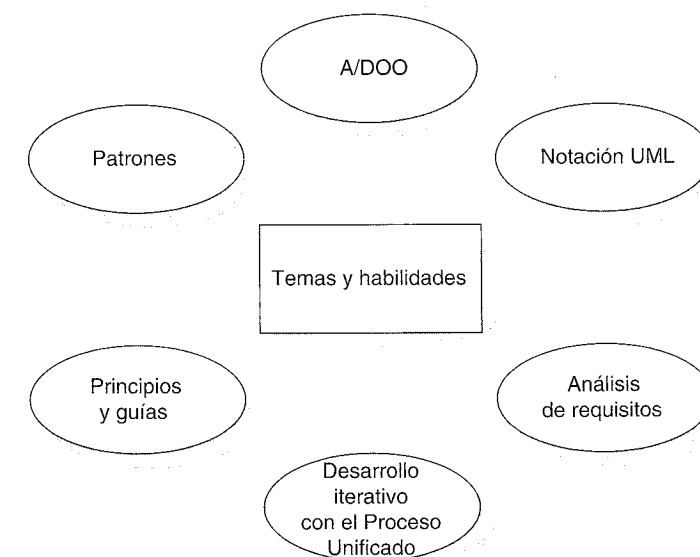


Figura 1.1. Temas y habilidades que se cubren.

Muchas otras habilidades son importantes

La construcción de software conlleva innumerables habilidades y pasos más allá del análisis de requisitos, el A/DOO, y la programación orientada a objetos. Por ejemplo, la ingeniería de usabilidad y el diseño de interfaces de usuario son claves para el éxito; de igual modo que el diseño de bases de datos.

Sin embargo, esta introducción se centra en el A/DOO, y no pretende cubrir todas las cuestiones del desarrollo de software. Es una parte de un dibujo más grande.

1.2. Asignación de responsabilidades

Hay muchas posibles actividades y artefactos en una introducción al A/DOO, y un gran número de principios y directrices. Suponga que debemos elegir una única habilidad práctica entre todos los temas aquí expuestos —una habilidad como una “isla desierta”—. ¿Cuál sería?

Una habilidad clave y fundamental en el A/DOO es la asignación cuidadosa de responsabilidades a los componentes software.

¿Por qué? Porque es una actividad que debe efectuarse —o mientras se dibuja un diagrama UML o programando— e influye fuertemente sobre la robustez, mantenimiento y reutilización de los componentes software.

Por supuesto, hay otras habilidades necesarias en el A/DOO, pero en esta introducción se hace hincapié en la asignación de responsabilidades porque suele ser una habi-

lidad que requiere esfuerzo el llegar a dominarla y al mismo tiempo tiene una importancia vital. En un proyecto real, un desarrollador podría no tener la oportunidad de abordar ninguna otra actividad de análisis o diseño —el proceso de desarrollo con “prisas por codificar”—. Pero incluso en esta situación, la asignación de responsabilidades es inevitable.

En consecuencia, los pasos de diseño de este libro se centran en los principios de asignación de responsabilidades.

Se presentan y aplican nueve principios fundamentales en el diseño de objetos y asignación de responsabilidades. Se organizan en una ayuda al aprendizaje denominada patrones GRASP.

1.3. ¿Qué es análisis y diseño?

El **Análisis** pone énfasis en una *investigación* del problema y los requisitos, en vez de ponerlo en una solución. Por ejemplo, si se desea un nuevo sistema de información informatizado para una biblioteca, ¿cómo se utilizará?

“Análisis” es un término amplio, es más adecuado calificarlo, como *análisis de requisitos* (un estudio de los requisitos) o *análisis de objetos* (un estudio de los objetos del dominio).

El **Diseño** pone énfasis en una *solución conceptual* que satisface los requisitos, en vez de ponerlo en la implementación. Por ejemplo, una descripción del esquema de una base de datos y objetos software. Finalmente, los diseños pueden ser implementados.

Como con el análisis, es más apropiado calificar el término como *diseño de objetos* o *diseño de bases de datos*.

El análisis y el diseño se han resumido en la frase *hacer lo correcto (análisis)*, y *hacerlo correcto (diseño)*.

1.4. ¿Qué son el análisis y el diseño orientados a objetos?

Durante el **análisis orientado a objetos**, se presta especial atención a encontrar y describir los objetos —o conceptos— en el dominio del problema. Por ejemplo, en el caso del sistema de información de la biblioteca, algunos de los conceptos son *Libro*, *Biblioteca*, y *Socio*.

Durante el **diseño orientado a objetos**, se presta especial atención a la definición de los objetos software y en cómo colaboran para satisfacer los requisitos. Por ejemplo, en el sistema de la biblioteca, un objeto software *Libro* podría tener un atributo *título* y un método *obtenerCapitulo* (ver Figura 1.2).

Por último, durante la implementación o programación orientada a objetos, los objetos de diseño se implementan, como la clase Java *Libro*.

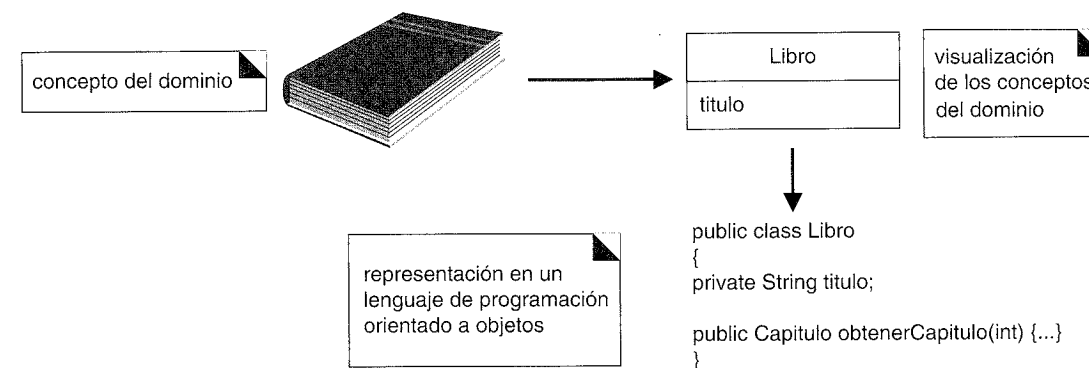


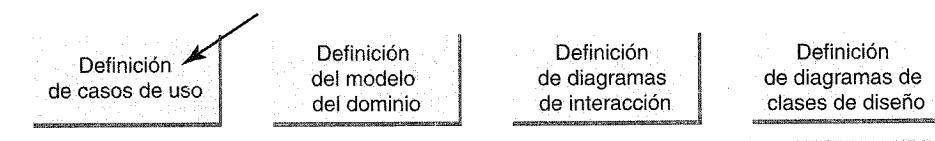
Figura 1.2. La orientación a objetos presta especial atención a la representación de los objetos.

1.5. Un ejemplo

Antes de sumergirnos en los detalles del análisis de requisitos y el A/DOO, esta sección presenta, de modo superficial, unos pocos pasos y diagramas claves, utilizando un ejemplo sencillo —un “juego de dados” en el que un jugador lanza dos dados—. Si el total es siete, gana; en otro caso, pierde.

Definición de los casos de uso

El análisis de requisitos podría incluir una descripción de los procesos del dominio relacionados, que podrían representarse como **casos de uso**.

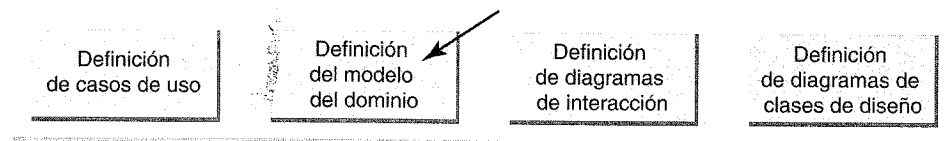


Los casos de uso no son artefactos orientados a objetos —son simplemente historias escritas—. Sin embargo, son una herramienta muy popular en análisis de requisitos y son una parte importante del Proceso Unificado. Por ejemplo, aquí está una versión breve del caso de uso *Jugar una Partida de Dados*:

Jugar una partida de dados: Un jugador recoge y lanza los dados. Si el valor de las caras de los dados suman siete, gana; en otro caso, pierde.

Definición de un modelo del dominio

La finalidad del análisis orientado a objetos es crear una descripción del dominio desde la perspectiva de la clasificación de objetos. Una descomposición del dominio conlleva una identificación de los conceptos, atributos y asociaciones que se consideran significativas. El resultado se puede expresar en un **modelo del dominio**, que se ilustra mediante un conjunto de diagramas que muestran los objetos o conceptos del dominio.



Por ejemplo, la Figura 1.3 muestra un modelo del dominio parcial.

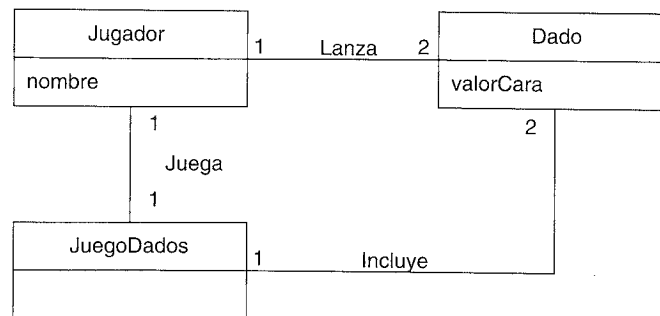


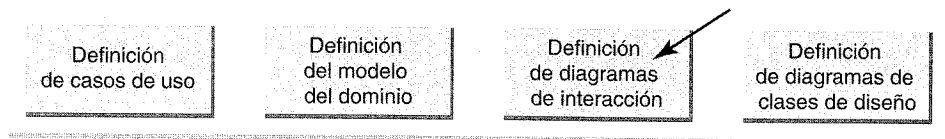
Figura 1.3. Modelo del dominio parcial del juego de dados.

Este modelo ilustra los conceptos importantes *Jugador*, *Dado* y *JuegoDados*, con sus asociaciones y atributos.

Nótese que un modelo del dominio no es una descripción de los objetos software, es una visualización de los conceptos en el dominio del mundo real.

Definición de los diagramas de interacción

La finalidad del diseño orientado a objetos es definir los objetos software y sus colaboraciones. Una notación habitual para ilustrar estas colaboraciones es el **diagrama de interacción**. Muestra el flujo de mensajes entre los objetos software y, por tanto, la invocación de métodos.



Por ejemplo, supongamos que se desea la implementación de un juego de dados. El diagrama de interacción de la Figura 1.4 ilustra los pasos esenciales del juego, enviando mensajes a las clases *JuegoDados* y *Dado*.

Nótese que aunque en el mundo real un *jugador* lanza los dados, en el diseño software el objeto *JuegoDados* "tira" los dados (es decir, envía mensajes a los objetos *Dado*). Los diseños de los objetos software y los programas se inspiran en los dominios del mundo real, pero *no* son modelos directos o simulaciones del mundo real.

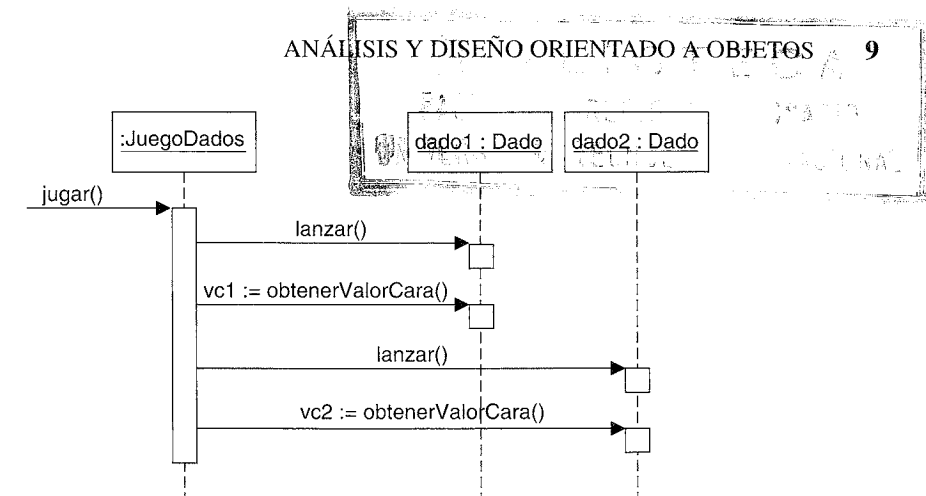
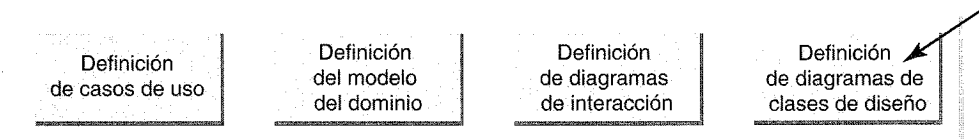


Figura 1.4. Diagrama de interacción que muestra los mensajes entre los objetos software.

Definición de los diagramas de clases de diseño

Además de la vista *dinámica* de las colaboraciones entre los objetos que se muestra mediante los diagramas de interacción, es útil crear una vista *estática* de las definiciones de las clases mediante un **diagrama de clases de diseño**.



Por ejemplo, en el juego de dados, un estudio del diagrama de interacción nos conduce al diagrama de clases de diseño parcial que se muestra en la Figura 1.5. Puesto que se envía el mensaje *jugar* al objeto *JuegoDados*, *JuegoDados* requiere un método *jugar*, mientras la clase *Dado* requiere los métodos *lanzar* y *obtenerValorCara*.

A diferencia del modelo de dominio, este diagrama no muestra conceptos del mundo real, sino clases software.

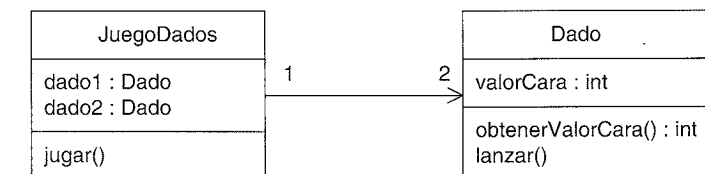


Figura 1.5. Diagrama de clases de diseño parcial.

Resumen

El juego de dados es un problema sencillo, presentado para centrar nuestra atención en unos pocos pasos y artefactos del análisis y diseño. Para no complicar la introducción, no se ha explicado toda la notación UML que se ha mostrado. En los siguientes capítulos se explorará el análisis y el diseño y estos artefactos con más detalle.

1.6. UML

Citando textualmente:

El Lenguaje Unificado de Modelado (UML) es un lenguaje para especificar, visualizar, construir y documentar los artefactos de los sistemas software, así como para el modelado del negocio y otros sistemas no software [OMG01].

UML se ha convertido en la notación visual estándar *de facto* y *de iure* para el modelado orientado a objetos. Comenzó como una iniciativa de Grady Booch y Jim Rumbaugh en 1994 para combinar las notaciones visuales de sus dos populares métodos —los métodos de Booch y OMT (*Object Modeling Technique*)—. Más tarde se les unió Ivar Jacobson, el creador del método Objectory, y el grupo comenzó a ser conocido como los *tres amigos*. Muchos otros contribuyeron a UML, quizás más notablemente Cris Kobryn, que lidera el proceso de refinamiento que todavía continúa.

UML fue adoptado en 1997 como estándar por el OMG (*Object Management Group*, organización que promueve estándares para la industria), y continúa siendo refinado en nuevas versiones.

Este libro no cubre cada pequeño aspecto de UML, que es una notación muy grande (algunos dicen, demasiado grande¹). Se centra en los diagramas que se utilizan con más frecuencia, las características utilizadas más comúnmente en esos diagramas, y la notación central que no es probable que cambie en futuras versiones de UML.

¿Por qué no veremos mucho UML durante unos pocos capítulos?

Éste no es simplemente un libro de notación UML, sino que trata de algo más amplio como es la aplicación de UML, patrones y un proceso iterativo en el contexto de desarrollo de software. UML se aplica principalmente durante el A/DOO, precedido, normalmente, por el análisis de requisitos. Por tanto, los capítulos iniciales presentan una introducción a las cuestiones importantes de casos de uso y análisis de requisitos, seguidos entonces de capítulos de A/DOO y más detalles de UML.

1.7. Lecturas adicionales

UML Distilled, de Martin Fowler, es un libro que ha tenido mucho éxito, y que merece la pena leer, que describe de forma concisa la notación UML más importante.

The Rational Unified Process-An Introduction de Philippe Kruchten es una introducción concisa y muy conocida al Proceso Unificado (y su refinamiento en el Proceso Unificado de Rational).

¹ Entre los objetivos de la versión 2.0 de UML se incluye la simplificación y reducción de la notación. Este libro presenta la parte de UML más utilizada que es muy probable que se mantenga tras la simplificación.

Para una discusión más detallada sobre la notación UML (versión 1.3), merecen la pena *The Unified Modeling Language Reference Manual* y *The Unified Modeling Language User Guide*, de Booch, Jacobson y Rumbaugh. Nótese que la intención de estos textos no era enseñar cómo hacer el modelado de objetos o el A/DOO —son referencias de la notación de los diagramas UML—.

Para una descripción de la versión actual de UML, es necesario la especificación on-line *OMG Unified Modeling Language Specification* en www.omg.org. Los trabajos de revisión de UML y las versiones que van a estar disponibles pronto se pueden encontrar en www.celigent.com/uml.

Hay muchos libros sobre patrones software, pero el libro que más ha influido en el área es el clásico *Design Patterns* de Gamma, Helm, Johnson y Vlissides. Es una lectura verdaderamente obligada para aquellos que estudian el diseño de objetos. Sin embargo, no es un texto introductorio y es mejor que se lea después de conocer bien los fundamentos del diseño y la programación de objetos.

DESARROLLO ITERATIVO Y EL PROCESO UNIFICADO

*Las personas son más importantes que cualquier proceso.
Buenas personas con un buen proceso siempre actuarán mejor
que buenas personas sin procesos.*

Grady Booch

Objetivos

- Explicar la motivación del orden y contenido de los capítulos siguientes.
 - Definir un proceso iterativo y adaptable.
 - Definir los conceptos fundamentales del Proceso Unificado.
-

Introducción

El desarrollo iterativo es un enfoque para el desarrollo de software que requiere un entrenamiento y poseer ciertos conocimientos, y juega un papel central en el modo en que se presenta el A/DOO en este libro. El Proceso Unificado es un ejemplo de proceso iterativo para proyectos que utilizan el A/DOO, y da forma a la presentación del libro. En consecuencia, es útil leer este capítulo para clarificar estos conceptos fundamentales y su influencia en la estructura del libro.

Este capítulo resume algunas ideas claves; el lector se puede dirigir al Capítulo 37 para una discusión más detallada del UP y las prácticas de los procesos iterativos.

De manera informal, un **proceso de desarrollo de software** describe un enfoque para la construcción, desarrollo y, posiblemente, mantenimiento del software. El **Proceso Unificado** [JBR99] se ha convertido en un proceso de desarrollo de software de gran

éxito para la construcción de sistemas orientados a objetos. En particular, se ha adoptado ampliamente el **Proceso Unificado de Rational** o **RUP** (*Rational Unified Process*) [Kruchten00], un refinamiento detallado del Proceso Unificado.

El Proceso Unificado (UP) combina las prácticas comúnmente aceptadas como “buenas prácticas”, tales como ciclo de vida iterativo y desarrollo dirigido por el riesgo, en una descripción consistente y bien documentada. Por tanto, se utiliza en este libro como ejemplo de proceso para introducir el A/DOO.

Este libro comienza con una introducción al UP por dos motivos:

1. El UP es un proceso *iterativo*. El desarrollo iterativo es una práctica de gran valor que influye en el modo en el que se introduce el A/DOO en este libro y en cómo debe ser aplicado.
2. Las prácticas del UP proporcionan una estructura organizada de ejemplo para discutir sobre cómo hacer —y cómo aprender— el A/DOO.

Este texto proporciona una introducción al UP, no lo aborda por completo. Se centra en las ideas y artefactos comunes relacionados con una introducción al A/DOO y el análisis de requisitos.

¿Y si no me interesa el UP?

El UP se utiliza como ejemplo de proceso con el que explorar el análisis de requisitos y el A/DOO, puesto que es necesario introducir el tema en el contexto de algún proceso, y el uso del UP (o el refinamiento del RUP) está relativamente bastante extendido. Además, el UP presenta actividades comunes y buenas prácticas. No obstante, las ideas centrales de este libro —como casos de uso y patrones de diseño— son independientes de cualquier proceso particular, y pueden aplicarse a muchos.

2.1. La idea más importante del UP: desarrollo iterativo

El UP fomenta muchas buenas prácticas, pero una destaca sobre las demás: el **desarrollo iterativo**. En este enfoque, el desarrollo se organiza en una serie de mini-proyectos cortos, de duración fija (por ejemplo, cuatro semanas) llamados **iteraciones**; el resultado de cada uno es un sistema que puede ser probado, integrado y ejecutado. Cada iteración incluye sus propias actividades de análisis de requisitos, diseño, implementación y pruebas.

El ciclo de vida iterativo se basa en la ampliación y refinamiento sucesivos del sistema mediante múltiples iteraciones, con retroalimentación cíclica y adaptación como elementos principales que dirigen para converger hacia un sistema adecuado. El sistema crece incrementalmente a lo largo del tiempo, iteración tras iteración, y por ello, este enfoque también se conoce como **desarrollo iterativo e incremental** (ver Figura 2.1).

Las primeras ideas sobre procesos iterativos se conocieron como desarrollo en espiral y desarrollo evolutivo [Boehm88, Gilb88].

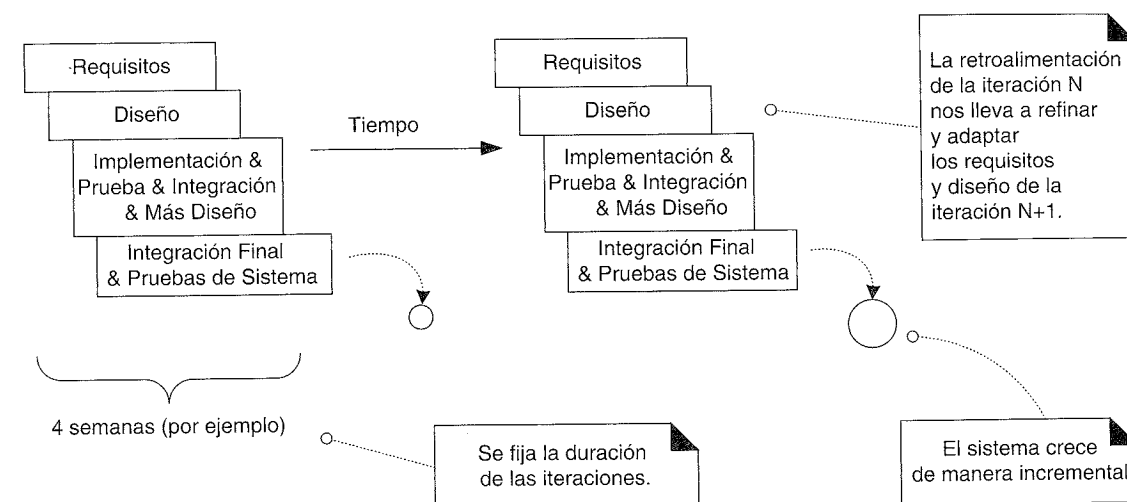


Figura 2.1. Desarrollo iterativo e incremental.

Ejemplo:

Como ejemplo (no como receta), en una iteración de dos semanas, a mitad de camino en un proyecto, quizás el lunes se dedique a distribuir y clarificar las tareas y requisitos de la iteración, mientras que una persona hace ingeniería inversa pasando el código de la última iteración a diagramas UML (mediante una herramienta CASE), e imprime y muestra los diagramas interesantes. El martes se dedica a diseñar por parejas en las pizarras, dibujando diagramas UML imprecisos que se capturan con cámaras digitales, y a escribir algo de pseudocódigo y notas de diseño. Durante los ocho días restantes, se dedica el tiempo a implementar, probar (unidad, aceptación, usabilidad...), ampliar el diseño, integrar, llevar a cabo construcciones diarias, pruebas de sistema y estabilización del sistema parcial. Otras actividades comprenden presentaciones y evaluaciones con el personal involucrado en el proyecto (*stakeholders*¹), y planificación de la siguiente iteración.

Nótese que en este ejemplo no hay prisa por codificar, ni una etapa de diseño más larga de lo que uno piensa que debería ser, en la que se pretende perfeccionar todos los detalles del diseño antes de programar. Se anticipa “un poco” del diseño con modelado basado en diagramas, utilizando dibujos UML realizados con rapidez y a grandes rasgos; quizás los desarrolladores dedican medio día, o un día entero, a trabajar diseñando en parejas.

El resultado de cada iteración es un sistema ejecutable, pero incompleto; no está preparado para ser puesto en producción. El sistema podría no estar listo para su puesta en producción hasta después de muchas iteraciones; por ejemplo, 10 ó 15.

La salida de una iteración *no* es un prototipo experimental o desechable, y el desarrollo iterativo no es prototipado. Más bien, la salida es un subconjunto con calidad de producción del sistema final.

Aunque, en general, cada iteración aborda nuevos requisitos y amplía el sistema incrementalmente, una iteración podría, ocasionalmente, volver sobre el software que ya existe y mejorarlo; por ejemplo, una iteración podría centrarse en mejorar el rendimiento de un subsistema, en lugar de extenderlo con nuevas características.

¹ N. del T.: Este término se refiere a todas las personas que de uno u otro modo están involucradas en un proyecto (directivos, usuarios, analistas, programadores...).

Aceptando los cambios: retroalimentación y adaptación

El subtítulo de un libro que trata el desarrollo iterativo es *Aceptar el Cambio* [Beck00]. Esta frase evoca una aptitud clave del desarrollo iterativo: En lugar de luchar contra el inevitable cambio que ocurre en el desarrollo de software intentando (normalmente sin éxito) especificar, congelar y “firmar” de manera completa y correcta a partir de un conjunto de requisitos fijos y diseñar antes de implementar, el desarrollo iterativo se basa en una aptitud de aceptación del cambio y la adaptación como motores inevitables y, de hecho, esenciales.

Esto no quiere decir que el desarrollo iterativo y el UP fomenten un proceso dirigido por “una adición de características” de manera incontrolada y reactiva. Los siguientes capítulos explorarán cómo el UP llega a un equilibrio entre la necesidad —por un lado— de llegar a un acuerdo y estabilizar un conjunto de requisitos, y —por otro lado— la realidad de los requisitos cambiantes, cuando el personal involucrado clarifica su visión o cambia el mercado.

Cada iteración conlleva la elección de un pequeño conjunto de requisitos y, rápidamente, diseñar, implementar y probar. En las primeras iteraciones, la elección de los requisitos y el diseño podrían no ser exactamente lo que se desea al final. Pero el acto de dar un pequeño paso con rapidez, antes de capturar todos los requisitos y que el diseño completo se haya definido de forma especulativa, nos lleva a una rápida retroalimentación —de los usuarios, desarrolladores y pruebas (tales como pruebas de carga y usabilidad)—.

Tener retroalimentación en una etapa temprana vale su peso en oro; más que las *especulaciones* sobre los requisitos y diseños correctos, la retroalimentación, a partir de la construcción y prueba realista de algo, aporta un conocimiento práctico y crucial, y una oportunidad de modificar o adaptar la comprensión de los requisitos o el diseño. Los usuarios finales tienen la oportunidad de ver rápidamente una parte del sistema y decir: “Sí, esto es lo que pedí, pero ahora que lo pruebo, lo que realmente quiero es algo un poco distinto”². Este proceso de “sí... pero” no es un signo de fallo; sino, ciclos estructurados frecuentes y tempranos de “sí... peros”, son un modo habilidoso de hacer progresar y descubrir qué es lo que tiene un valor real para el personal involucrado. Si, como se ha mencionado, esto no es un consentimiento, un desarrollo caótico y reactivo en el que los desarrolladores cambien continuamente de dirección —es posible llegar a un término medio—.

Además de clarificar los requisitos, actividades como la prueba de carga probarán si el diseño y la implementación parcial están en el camino correcto, o si en la siguiente iteración, se necesita un cambio en la arquitectura básica. Cuanto antes se resuelvan y *prueben* las decisiones de diseño críticas y arriesgadas mejor —el desarrollo iterativo proporciona los mecanismos para esto—.

En consecuencia, el trabajo se desarrolla a lo largo de una serie de ciclos estructurados de construir-retroalimentar-adaptar. No sorprende que la desviación del sistema del

² O más probable: “¡No entendí lo que quería!”

“verdadero camino” (en términos de sus requisitos y diseño finales) en las primeras iteraciones será mayor que en las últimas. A lo largo del tiempo, el sistema converge hacia este camino, como se ilustra en la Figura 2.2.

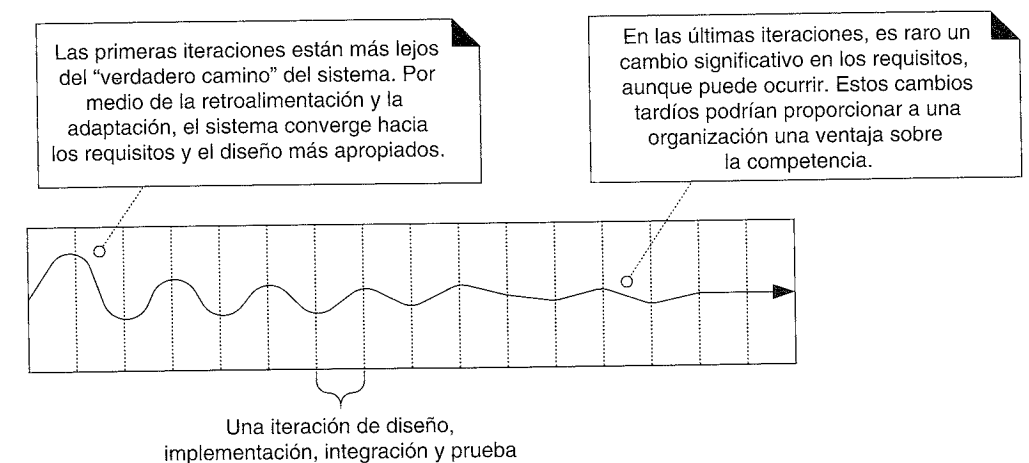


Figura 2.2. La retroalimentación iterativa y la adaptación nos conducen hacia el sistema deseado. La inestabilidad de los requisitos y el diseño disminuyen a lo largo del tiempo.

Beneficios del desarrollo iterativo

Los beneficios del desarrollo iterativo incluyen:

- Mitigación tan pronto como sea posible de riesgos altos (técnicos, requisitos, objetivos, usabilidad y demás)
- Progreso visible en las primeras etapas.
- Una temprana retroalimentación, compromiso de los usuarios y adaptación, que nos lleva a un sistema refinado que se ajusta más a las necesidades reales del personal involucrado.
- Gestión de la complejidad; el equipo no se ve abrumado por la “parálisis del análisis” o pasos muy largos y complejos.
- El conocimiento adquirido en una iteración se puede utilizar metódicamente para mejorar el propio proceso de desarrollo, iteración a iteración.

Longitud de una iteración y fijación de la duración

El UP (y desarrolladores con experiencia en aplicar procesos iterativos) recomiendan que la longitud de una iteración sea de dos a seis semanas. Pasos pequeños, rápida retroalimentación, y adaptación son las ideas fundamentales del desarrollo iterativo; iteraciones largas destruyen la motivación principal del desarrollo iterativo e incrementan el riesgo del proyecto. Menos de dos semanas y es difícil completar el trabajo suficiente para obtener resultados significativos y retroalimentación; más de seis u ocho semanas, y la complejidad se hace bastante abrumadora, y se retrasa la retroalimentación. Con iteraciones muy largas pierde su sentido el desarrollo iterativo. Lo corto es bueno.

Una idea clave es que se **fija la duración** de las iteraciones. Por ejemplo, si se elige que la siguiente iteración dure cuatro semanas, entonces el sistema parcial debería integrarse, probarse y estabilizarse en la fecha planificada —los retrasos son frustrantes—. Si parece que será difícil cumplir con el plazo fijado, la respuesta recomendada es eliminar tareas o requisitos de la iteración, e incluirlos en una iteración posterior, más que retrasar la fecha de terminación prevista. El Capítulo 37 resume los motivos para fijar la duración.

Equipos muy grandes (por ejemplo, varios cientos de desarrolladores) podrían requerir iteraciones de más de seis semanas para compensar los costes fijos de coordinación y comunicación; pero no se recomienda que sea más de tres a seis semanas. Por ejemplo, la exitosa sustitución de los noventa del sistema de control aéreo canadiense, se desarrolló siguiendo un ciclo de vida iterativo y otras prácticas del UP. Necesitó 150 programadores que se organizaron en iteraciones de seis meses³. Pero, nótese que incluso en el caso de una iteración de proyecto global de seis meses, un equipo encargado de un subsistema formado por 10 ó 20 desarrolladores puede dividir su trabajo en una serie de seis iteraciones de un mes.

Una iteración de seis meses es la excepción, en el caso de grandes equipos, no la regla. Reiterando lo dicho, el UP recomienda que la duración de una iteración sea entre dos y seis semanas.

2.2. Conceptos y buenas prácticas del UP adicionales

La idea fundamental para apreciar y utilizar el UP es el desarrollo iterativo, fijando iteraciones cortas, y adaptable.

Otra idea del UP implícita, pero muy importante, es el uso de las tecnologías de objetos, entre las que se encuentra el A/DOO y la programación orientada a objetos.

Algunos conceptos claves y buenas prácticas del UP son:

- Abordar cuestiones de alto riesgo y muy valiosas en las primeras iteraciones
- Involucrar continuamente a los usuarios para evaluación, retroalimentación y requisitos
- Construir en las primeras iteraciones una arquitectura que constituya un núcleo central consistente
- Verificar la calidad continuamente; pruebas muy pronto, con frecuencia y de manera realista
- Aplicar casos de uso
- Modelar software visualmente (con UML)
- Gestionar los requisitos con cuidado
- Manejar peticiones de cambio y gestión de configuraciones

El lector puede acudir al Capítulo 37 para una descripción más detallada de estas prácticas.

³ Philippe Kruchten, quien también dirigió el desarrollo del RUP, trabajó como arquitecto jefe en el proyecto.

2.3. Las fases del UP y términos orientados a la planificación

Un proyecto UP organiza el trabajo y las iteraciones en cuatro fases fundamentales:

- 1. Inicio:** visión aproximada, análisis del negocio, alcance, estimaciones imprecisas.
- 2. Elaboración:** visión refinada, implementación iterativa del núcleo central de la arquitectura, resolución de los riesgos altos, identificación de más requisitos y alcance, estimaciones más realistas.
- 3. Construcción:** implementación iterativa del resto de requisitos de menor riesgo y elementos más fáciles, preparación para el despliegue.
- 4. Transición:** pruebas beta, despliegue.

Estas fases se definen de una manera más completa en los capítulos siguientes.

Esto *no* se corresponde con el antiguo ciclo de vida “en cascada” o secuencial, en el que primero se definían todos los requisitos y, después, se realizaba todo, o la mayoría, del diseño.

La fase de Inicio no es una fase de requisitos; sino una especie de fase de viabilidad, donde se lleva a cabo sólo el estudio suficiente para decidir si continuar o no.

De igual modo, la fase de Elaboración no es la fase de requisitos o de diseño; sino que es una fase donde se implementa, de manera iterativa, la arquitectura que constituye el núcleo central y se mitigan las cuestiones de alto riesgo.

La Figura 2.3 ilustra los términos orientados a la planificación comunes del UP. Nótese que un ciclo de desarrollo (que termina con el lanzamiento de un sistema a producción) se compone de muchas iteraciones.

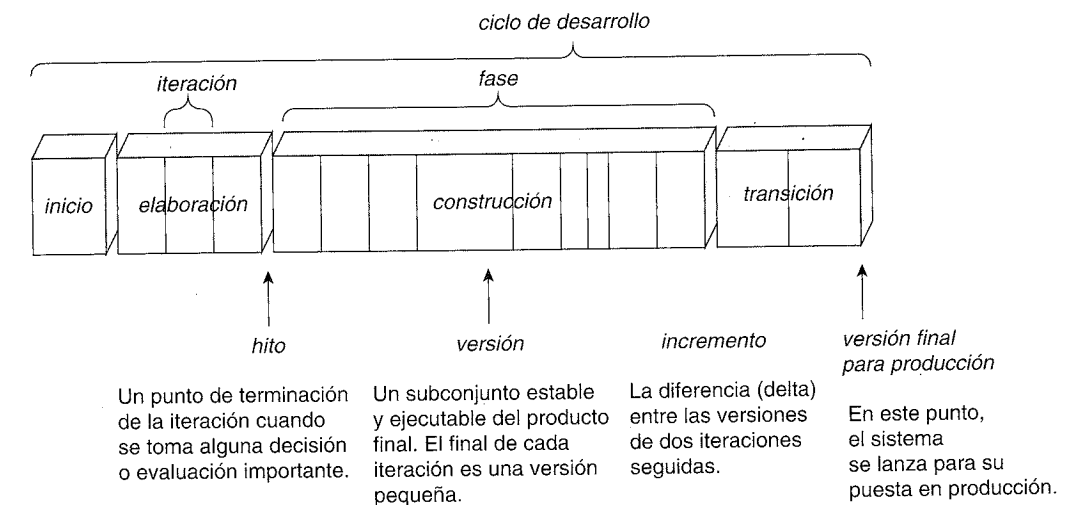


Figura 2.3. Términos orientados a la planificación en el UP.

2.4. Las disciplinas del UP (eran flujos de trabajo)

El UP describe actividades de trabajo, como escribir casos de uso, en **disciplinas** (llamadas originalmente **flujos de trabajo**)⁴. Informalmente, una disciplina es un conjunto de actividades (y artefactos relacionados) en un área determinada, como las actividades en el análisis de requisitos. En el UP, un **artefacto** es el término general para cualquier producto del trabajo: código, gráficos Web, esquema de base de datos, documentos de texto, diagramas, modelos, etcétera.

Hay varias disciplinas en el UP; este libro se centra en algunos artefactos de las siguientes tres:

- **Modelado del Negocio.** Cuando se desarrolla una única aplicación, esto incluye el modelado de los objetos del dominio. Cuando se está haciendo análisis del negocio a gran escala o reingeniería de procesos del negocio, esto incluye el modelado dinámico de los procesos del negocio de toda la empresa.
- **Requisitos.** Análisis de los requisitos para una aplicación, como escritura de casos de uso e identificación de requisitos no funcionales.
- **Diseño.** Todos los aspectos de diseño, incluyendo la arquitectura global, objetos, bases de datos, red y cosas parecidas.

Una lista más extensa de las disciplinas del UP se muestra en la Figura 2.4

En el UP, **Implementación** significa programar y construir el sistema, no despliegue. La disciplina **Entorno** se refiere a establecer las herramientas y adaptar el proceso al proyecto —esto es organizar la herramienta y el entorno del proceso—.

Disciplinas y fases

Como se ilustra en la Figura 2.4, durante una iteración, el trabajo se desarrolla en la mayoría o todas las disciplinas. Sin embargo, el esfuerzo relativo en estas disciplinas cambia a lo largo del tiempo. Las primeras iteraciones, naturalmente, tienden a aplicar un esfuerzo relativo mayor a los requisitos y al diseño, y en las posteriores disminuye, cuando los requisitos y el diseño central se estabilizan, mediante un proceso de retroalimentación y adaptación.

Relacionando esto con las fases del UP (inicio, elaboración...), la Figura 2.5 muestra el esfuerzo relativo de cambio con respecto a las fases; por favor, nótese que son una sugerencia, no es literal. En la elaboración, por ejemplo, las iteraciones tienden a tener un nivel relativamente alto de trabajo de requisitos y diseño, aunque, sin ninguna duda, también algo de implementación. Durante la construcción, se le da una importancia mayor a la implementación y más ligera al análisis de requisitos.

⁴ En el 2001, el antiguo término del UP “flujo de trabajo” se substituyó por el nuevo término “disciplina” para armonizar con un esfuerzo de estandarización internacional denominado OMG SPEM; debido al significado anterior en el UP, muchos continúan utilizando el término flujo de trabajo para referirse a disciplina, aunque no es estrictamente correcto. El término “flujo de trabajo” comenzó a tener un significado nuevo pero ligeramente distinto en el UP: en un proyecto particular, es una secuencia particular de actividades (quizás *entre* disciplinas) —un flujo de trabajo—.

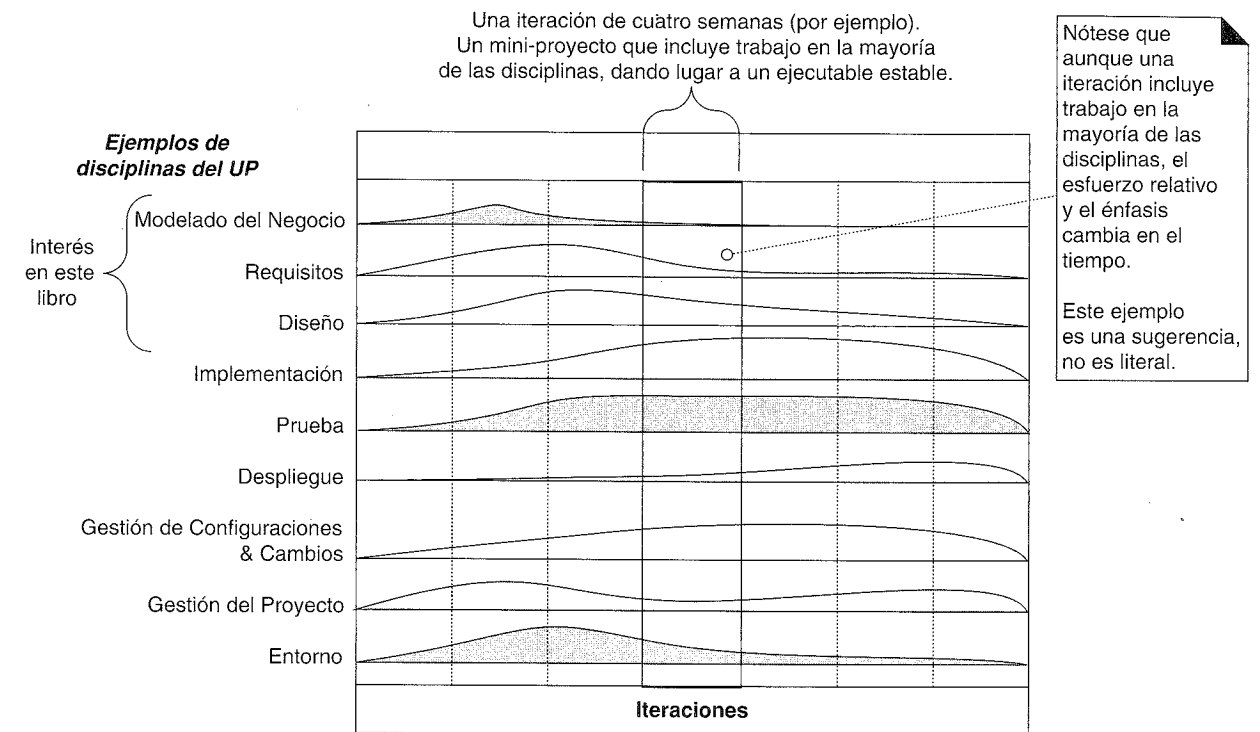


Figura 2.4. Disciplinas del UP⁵.

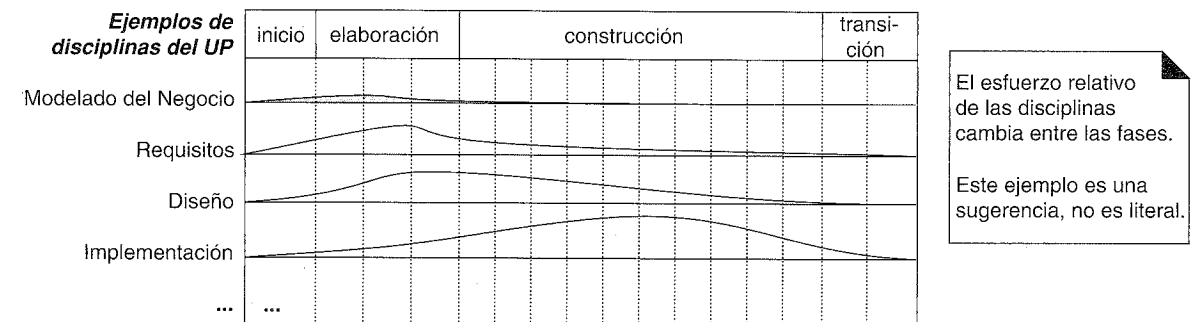


Figura 2.5. Disciplinas y fases.

Estructura del libro y fases y disciplinas del UP

Con respecto a las fases y disciplinas, ¿cuál es el enfoque del caso de estudio?

Respuesta:

El caso de estudio presta una atención especial a las fases de inicio y elaboración. Se centra en algunos artefactos de las disciplinas del Modelado del Negocio, Requisitos y Diseño, donde se aplican, fundamentalmente, el análisis de requisitos, el A/DOO, los patrones y UML.

⁵ Diagrama adaptado del producto RUP.

Los primeros capítulos presentan las actividades de la fase de inicio; los capítulos posteriores exploran varias iteraciones en la fase de elaboración. La siguiente lista y la Figura 2.6 describen la organización con respecto a las fases del UP.

- 1. Los capítulos de la fase de inicio introducen los temas esenciales del análisis de requisitos.
- 2. La iteración 1 presenta los fundamentos del A/DOO y cómo asignar reponsabilidades a los objetos.
- 3. La iteración 2 está enfocada al diseño de objetos, en concreto a introducir algunos de los “patrones de diseño” más utilizados.
- 4. La iteración 3 presenta una variedad de temas, como el análisis de la arquitectura y el diseño de frameworks.

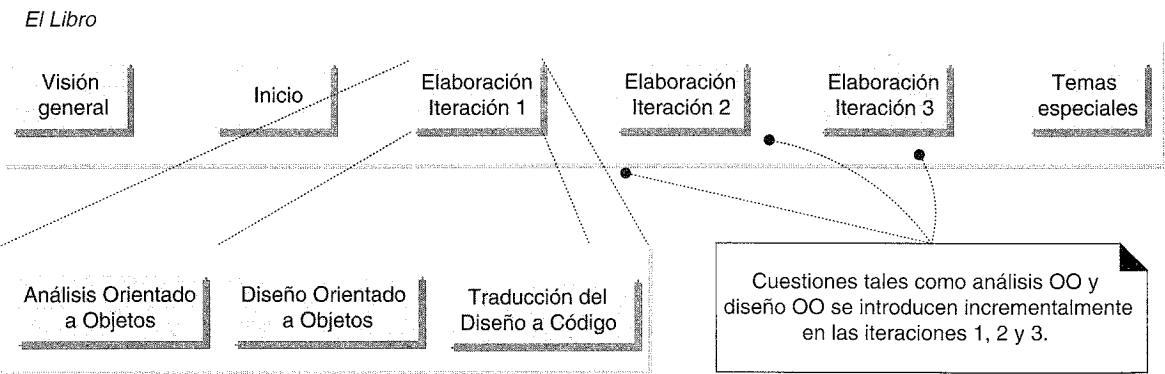


Figura 2.6. La organización del libro está relacionada con las fases e iteraciones del UP.

2.5. Adaptación del proceso y el Marco de Desarrollo

Artefactos opcionales

Algunas prácticas y principios del UP deben seguirse siempre, como el desarrollo iterativo y dirigido por el riesgo, y la verificación continua de la calidad.

Sin embargo, es importante saber que en el UP todas las actividades y artefactos (modelos, diagramas, documentos...) son *opcionales* —bueno, ¡quizás el código no!—. El conjunto de artefactos posibles del UP debería entenderse como un conjunto de medicinas en una farmacia. Exactamente igual que uno no toma medicinas indiscriminadamente, sino que las elige según la dolencia, en un proyecto UP, un equipo debería seleccionar un pequeño subconjunto de artefactos que sirvan para tratar sus problemas y necesidades particulares. En general, centrarse en un *pequeño* conjunto de artefactos que demuestran tener un gran valor práctico.

El Marco de Desarrollo

La elección de los artefactos del UP para un proyecto podría recogerse en un documento breve denominado **Marco de Desarrollo** (un artefacto en la disciplina Entorno). Por

ejemplo, la Tabla 2.1 podría ser el Marco de Desarrollo que describe los artefactos para el caso de estudio “Proyecto NuevaEra” presentado en este libro.

Los capítulos siguientes describen la creación de algunos de estos artefactos como el Modelo del Dominio, Modelo de Casos de Uso y el Modelo de Diseño.

Los artefactos de ejemplo que se presentan en este caso de estudio de ningún modo son suficientes, o adecuados, para todos los proyectos. Por ejemplo, un sistema de control de una máquina encontraría útil realizar muchos diagramas de estados. Un sistema de comercio electrónico basado en el web podría centrar su atención en los prototipos de interfaz de usuario. Un proyecto de desarrollo nuevo, en un campo en el que no se tenga experiencia tendría necesidades muy diferentes, en cuanto a los artefactos de diseño, de las de un proyecto de integración de sistemas.

Tabla 2.1 Ejemplo de Marco de Desarrollo de artefactos UP. c-comenzar; r-refinar

Disciplina	Artefacto Iteración →	Inicio I1	Elab. E1...En	Const. C1...Cn	Trans. T1...T2
Modelado del Negocio	Modelo del Dominio		c		
Requisitos	Modelo de Casos de Uso	c	r		
	Visión	c	r		
	Especificación Complementaria	c	r		
	Glosario	c	r		
Diseño	Modelo de Diseño		c	r	
	Documento de Arquitectura SW		c	r	
	Modelo de Datos		c	r	
Implementación	Modelo de Implementación		c	r	r
Gestión del Proyecto	Plan de Desarrollo SW	c	r	r	r
Pruebas	Modelo de Pruebas		c	r	
Entorno	Marco de Desarrollo	c	r		

2.6. El UP ágil

Los metodologistas distinguen entre procesos pesados y ligeros, y procesos predictivos y adaptables. Un **proceso pesado** es un término peyorativo, que pretende sugerir un proceso con las siguientes cualidades [Fowler00]:

- Muchos artefactos creados en un ambiente burocrático.
- Rigidez y control.
- Planificación detallada, muy larga y elaborada.
- Predictivo más que adaptable.

Un **proceso predictivo** es aquel que intenta planificar y predecir en detalle las actividades y asignación de recursos (personal) en un intervalo relativamente largo de tiempo, tal como la totalidad de un proyecto. Los procesos predictivos normalmente si-

guen un ciclo de vida en “cascada” o secuencial —primero, definición de todos los requisitos; segundo, definición del diseño detallado; y tercero, implementación—. Frente a esto, un **proceso adaptable** es aquel que acepta el cambio como motor inevitable y fomenta la adaptación flexible; normalmente siguen un ciclo de vida iterativo. Un **proceso ágil** implica un proceso adaptable y ligero, listo para responder rápidamente a las necesidades cambiantes.

La intención de los autores del UP no era que fuese pesado o predictivo, aunque su amplio conjunto de actividades y artefactos opcionales, comprensiblemente, ha llevado a algunos a tener esta impresión. Más bien, la intención fue que se adoptara y aplicara con el espíritu de un proceso ágil —UP ágil—. Algunos ejemplos de cómo se pone esto en práctica:

- Optar por un conjunto pequeño de actividades y artefactos del UP. Algunos proyectos se beneficiarán más que otros, pero, en general, manténgalo simple.
- Puesto que el UP es iterativo, no se completan los requisitos y diseños antes de la implementación. Surgen de modo adaptable a lo largo de una serie de iteraciones, basadas en la retroalimentación.
- No hay un plan *detallado* para todo el proyecto. Hay un plan de alto nivel (denominado el **Plan de Fase**) que estima la fecha de terminación del proyecto y otros hitos importantes, pero no detalla los pasos de grano fino de estos hitos. Un plan detallado (llamado **Plan de Iteración**) sólo planifica con gran detalle una iteración por adelantado. La planificación detallada se lleva a cabo de manera adaptable de iteración en iteración. En el Capítulo 36 se pueden encontrar algunos comentarios sobre la planificación de proyectos iterativos y la justificación de este enfoque.

El caso de estudio hace hincapié en un número relativamente pequeño de artefactos y en el desarrollo iterativo, en el espíritu de un UP ágil.

2.7. El ciclo de vida “en cascada” secuencial

A diferencia del ciclo de vida iterativo del UP, una antigua alternativa era el ciclo de vida “en cascada”, lineal o secuencial [Royce70]. En su acepción habitual, definía los pasos más o menos de la siguiente forma:

1. Determinar, registrar y acordar un conjunto de requisitos completo y fijo.
2. Diseñar un sistema basado en estos requisitos.
3. Implementar en base al diseño.

Un estudio de dos años, presentado en el *MIT Sloan Management Review*, sobre los proyectos software con éxito, identificó cuatro factores comunes para el éxito; el primero de la lista era el desarrollo iterativo, en lugar de un proceso en cascada [MacCormack01]⁶.

⁶ Los otros eran: 2) al menos una incorporación diaria de nuevo código para la construcción del sistema completo, y rápida retroalimentación de los cambios del diseño (a través de las pruebas); 3) equipo con experiencia en expedir múltiples productos; y 4) centrarse pronto en la construcción y prueba de una arquitectura consistente. Tres de estos cuatro factores son prácticas explícitas en el UP.

Una descripción breve de estos problemas, y cómo pueden mitigarse mediante el desarrollo iterativo, se presentará en el Capítulo 37.

2.8. No se entendió el UP cuando...

Aquí presentamos algunos signos que indican que no se ha entendido lo que significa adoptar el UP y el desarrollo iterativo con la idea de agilidad que se pretende en el UP.

- Se piensa que inicio = requisitos, elaboración = diseño, y construcción = implementación (esto es, imponiendo un ciclo de vida en espiral sobre el UP).
- Se piensa que el objetivo de la elaboración es definir modelos de manera completa y cuidadosa, los cuales se traducen a código durante la construcción.
- Se intenta definir la mayoría de los requisitos antes de comenzar el diseño o la implementación.
- Se intenta definir la mayoría del diseño antes de comenzar a implementar; se intenta definir completamente y acordar una arquitectura antes de programar y probar iterativamente.
- Se dedica mucho tiempo a realizar trabajo sobre los requisitos y el diseño antes de comenzar a programar.
- Se cree que una iteración adecuada es de cuatro meses de duración, en lugar de cuatro semanas (excluyendo proyectos de cientos de desarrolladores).
- Se piensa que realizar los diagramas UML y las actividades de diseño constituyen el momento para definir diseños y modelos de manera completa y precisa con gran detalle, y se cree que programar es una simple traducción mecánica de éstos en código.
- Se piensa que adoptar el UP significa hacer muchas de las actividades posibles y crear muchos documentos, y se piensa o experimenta el UP como un proceso formal y exigente con muchos pasos que seguir.
- Se intenta planificar un proyecto en detalle desde el principio hasta el final; intenta predecir, de manera especulativa, todas las iteraciones y lo que debería ocurrir en cada una de ellas.
- Se quieren planes y estimaciones creíbles para los proyectos antes de que termine la fase de elaboración.

2.9. Lecturas adicionales

Una introducción al UP y su refinamiento en el RUP, que merece la pena leer es *The Rational Unified Process-An Introduction* de Philippe Kruchten, el arquitecto líder del RUP.

Se puede encontrar una descripción del UP original en *The Unified Software Development Process* de Jacobson, Booch y Rumbaugh. Merece la pena estudiarlo, pero se recomienda primero la introducción de Kruchten puesto que es más pequeña y concisa, y el RUP actualiza y refina el original UP.

Rational Software vende un producto basado en el web con la documentación del RUP on-line, el cual permite conocer con todo detalle todas las actividades y artefactos del RUP y las plantillas para la mayoría de los artefactos. En el Capítulo 37 se presentará una breve discusión. Una organización puede abordar un proyecto UP utilizando únicamente mentores y libros como fuentes de aprendizaje, pero algunos encuentran la documentación del RUP una ayuda útil para el aprendizaje y el proceso.

Las actividades del UP también se describen de manera poco estricta en una serie de libros editados por Ambler y Constantine (por ejemplo, *The Unified Process: Elaboration Phase* [Ambler00]). Estos libros contienen artículos publicados a lo largo de los años en la revista *Software Development*, clasificados en sus respectivas fases y actividades en términos de una taxonomía del UP. Nótese que los artículos no se escribieron originalmente para el UP, aunque, sin lugar a dudas contienen consejos útiles. También se advierte un pequeño error en las series: describen la fase de elaboración del UP como una fase en la que se crean prototipos desechables, de este modo reduce la necesidad para prestar atención a cuidar la programación o el diseño. Esto no es exacto; durante la elaboración se crean diseños y código con calidad de producción (aunque parcial). Ambler reconoce la imprecisión y puede corregirlo en las ediciones siguientes⁷.

Para obtener información sobre otros métodos ágiles se recomienda la serie de libros **Extreme Programming** (XP) [Beck00, BF00, JAH00], tales como *Extreme Programming Explained*. Algunas de las prácticas de XP se mencionan en capítulos posteriores. La mayoría de las prácticas de XP (tales como programar probando primero y desarrollo iterativo) son compatibles —o idénticas— a las prácticas del UP, y recomendando su adopción en un proyecto UP. Nótese que XP no inventó (ni pidió que se hiciese) el desarrollo iterativo, con iteraciones cortas de duración fija, y adaptable, que ha sido una práctica del UP y otros métodos iterativos durante años. Dos diferencias destacables —no es una lista completa— entre el UP y XP son: 1) el UP recomienda escribir de manera incremental los casos de uso y un documento de requisitos no funcionales (XP no); y, 2) el UP recomienda dedicar más tiempo a realizar los diagramas del diseño visual (como medio día o un día entero) cerca del comienzo de una iteración, antes de la programación. Los líderes de XP recomiendan muy poco, unos 30 minutos.

Highsmith justifica el valor del desarrollo iterativo en *Adaptive Software Development* [Highsmith00].

⁷ Ambler, comunicación privada.

CASO DE ESTUDIO: EL SISTEMA DE PUNTO DE VENTA NUEVAERA

Pocas cosas son más difíciles de soportar que un buen ejemplo.

Mark Twain

Introducción

Este capítulo describe brevemente el caso de estudio. Si se conoce el dominio del problema, podría saltarse. De hecho, se eligió este problema porque es familiar, pero suficientemente rico, con problemas interesantes de arquitectura y diseño, y, por tanto, nos permite concentrarnos en cómo llevar a cabo el análisis y diseño, en lugar de explicar el problema y el dominio.

3.1. El sistema de punto de venta NuevaEra

El caso de estudio es el sistema de punto de venta (PDV) NuevaEra. En este dominio del problema, en apariencia sencillo, veremos que hay requisitos muy interesantes y problemas de diseño que solucionar. Además, es un problema real; las organizaciones realmente escriben sistemas PDV utilizando la tecnología de objetos.

Un sistema PDV es una aplicación informática utilizada (en parte) para registrar ventas y realizar pagos; normalmente se utiliza en tiendas. Incluye componentes hardware, como un ordenador y un lector de códigos de barras, y software para ejecutar el sistema. Interactúa con varias aplicaciones de servicios, como un servicio de cálculo de impuestos y un control de inventario, de terceras partes. Estos sistemas deben ser, relativamente, tolerantes a fallos; es decir, incluso si los servicios remotos no están disponibles temporalmente (como el sistema de inventario), todavía deben ser capaces de

capturar las ventas y gestionar, al menos, los pagos en efectivo (de manera que no se impida que el negocio funcione de manera adecuada).

Un sistema PDV, progresivamente, debe soportar múltiples y variados terminales e interfaces del lado del cliente, como un terminal con un navegador Web de una arquitectura “cliente delgado” (*thin client*), un ordenador personal normal con una interfaz gráfica de usuario hecha con las clases Swing de Java, entrada de datos mediante una pantalla táctil, PDAs inalámbricos, etcétera.

Además, estamos creando un sistema PDV comercial que se venderá a diferentes clientes con necesidades dispares en términos de procesamiento de reglas del negocio. Cada cliente deseará un conjunto exclusivo de lógica a ejecutar en ciertos puntos predecibles en escenarios de uso del sistema, como cuando se inicia una venta o cuando se añade una nueva línea. Por tanto, necesitaremos un mecanismo para proporcionar esta flexibilidad y personalización.

Utilizando una estrategia de desarrollo iterativo, vamos a realizar las fases de requisitos y análisis, diseño e implementación orientados a objetos.

3.2. Capas arquitectónicas y el énfasis del caso de estudio

Un sistema de información orientado a objetos típico se diseña en función de varias capas arquitectónicas o subsistemas (ver Figura 3.1). A continuación, presentamos algunos ejemplos, no una lista completa:

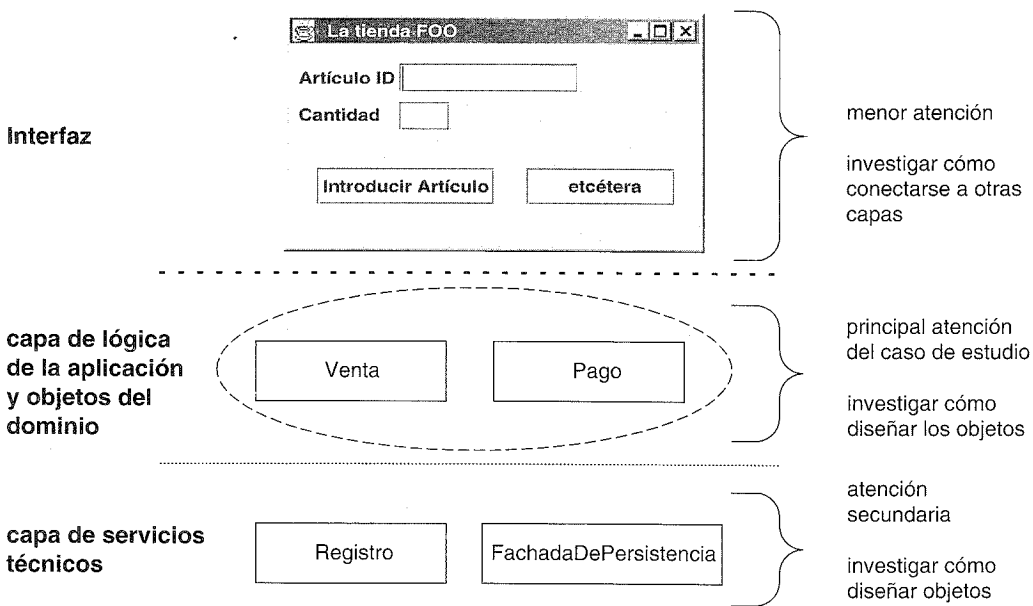


Figura 3.1. Capas y objetos de ejemplo en un sistema orientado a objetos, y el enfoque del caso de estudio.

- **Interfaz de Usuario:** interfaz gráfica; ventanas.
- **Lógica de la aplicación y Objetos del Dominio:** objetos software que representan conceptos del dominio (por ejemplo una clase software denominada *Venta*) que satisfacen los requisitos de la aplicación.
- **Servicios técnicos:** objetos de propósito general y subsistemas que proporcionan servicios técnicos de apoyo, como conexión con una base de datos o registrar los errores. Normalmente, estos servicios son independientes de la aplicación y se pueden reutilizar entre varios sistemas.

El A/DOO es, por lo general, más relevante para modelar la lógica de la aplicación y las capas de servicios técnicos.

El caso de estudio NuevaEra se centra principalmente en los objetos del dominio del problema, asignándoles responsabilidades para satisfacer los requisitos de la aplicación. El diseño orientado objetos también se aplica para crear un subsistema de servicio técnico para interactuar con una base de datos.

En este enfoque de diseño, la capa de interfaz de usuario tiene muy poca responsabilidad; se dice que es *delgada*. Las ventanas *no* contienen código que ejecute o procese la lógica de la aplicación, sino que, las peticiones de realización de tareas se envían a otras capas.

3.3. Estrategia del libro: aprendizaje y desarrollo iterativo

Este libro se organiza para seguir una estrategia de desarrollo iterativo. El A/DOO se aplica al sistema PDV NuevaEra en varias iteraciones; la primera iteración es para determinar algunas funciones básicas. Las iteraciones posteriores expanden la funcionalidad del sistema (ver Figura 3.2). Junto con el desarrollo iterativo, se introduce incrementalmente y de manera iterativa, la *presentación* de las cuestiones del análisis y diseño, la notación UML y los patrones. En la primera iteración, se presenta un núcleo básico de cuestiones de análisis, diseño y notación. La segunda iteración amplía con nuevas ideas, la notación UML y los patrones. Y, de igual modo, la tercera iteración.

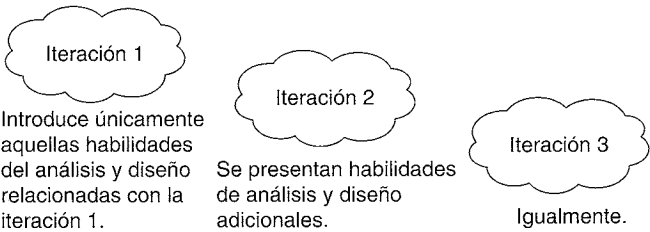


Figura 3.2. El camino de aprendizaje sigue iteraciones.